

Paxos Made Insanely Simple

Hein Meling

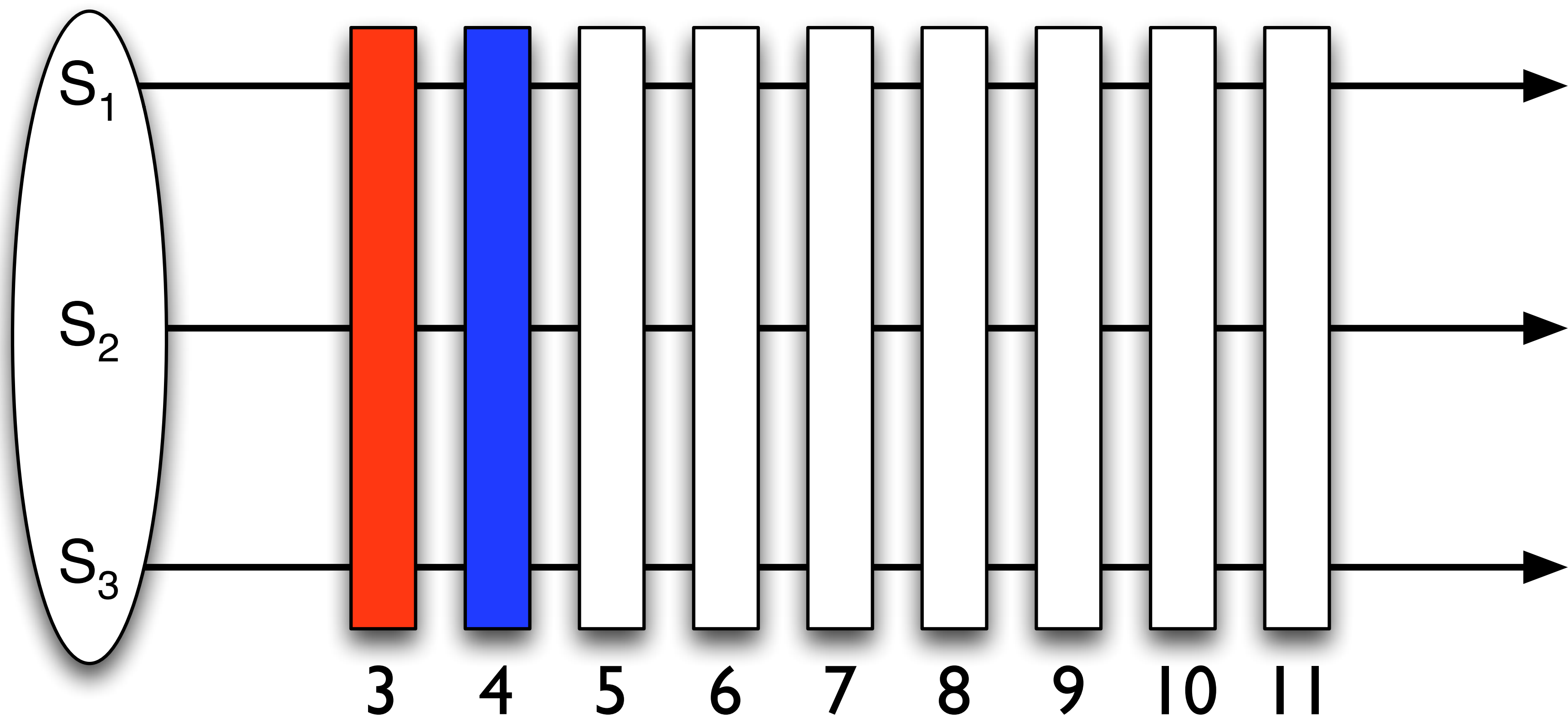


University of
Stavanger

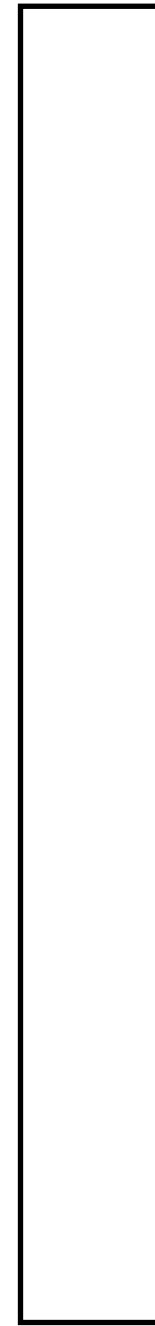
Paxos

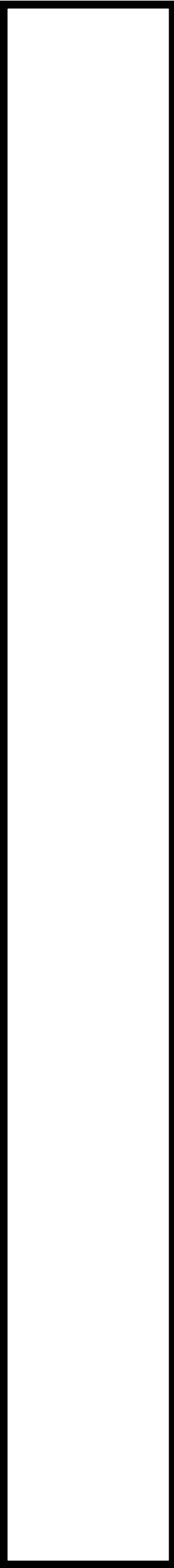
(Multi-decree Paxos)

Sequence of Slots



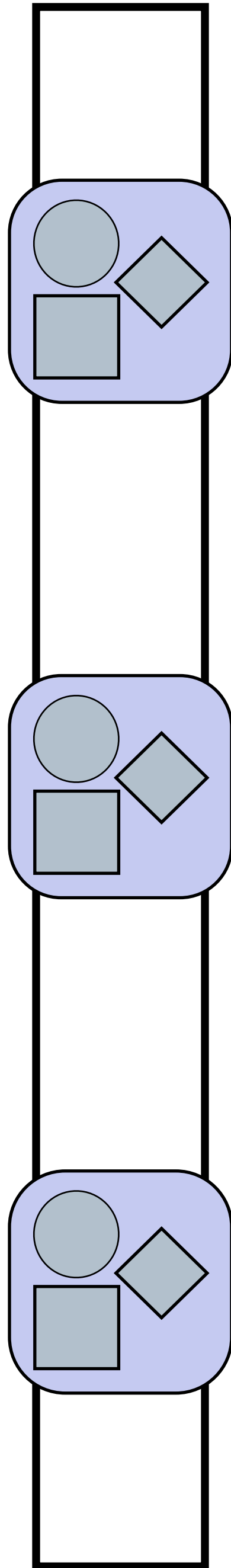
Single-decree Paxos



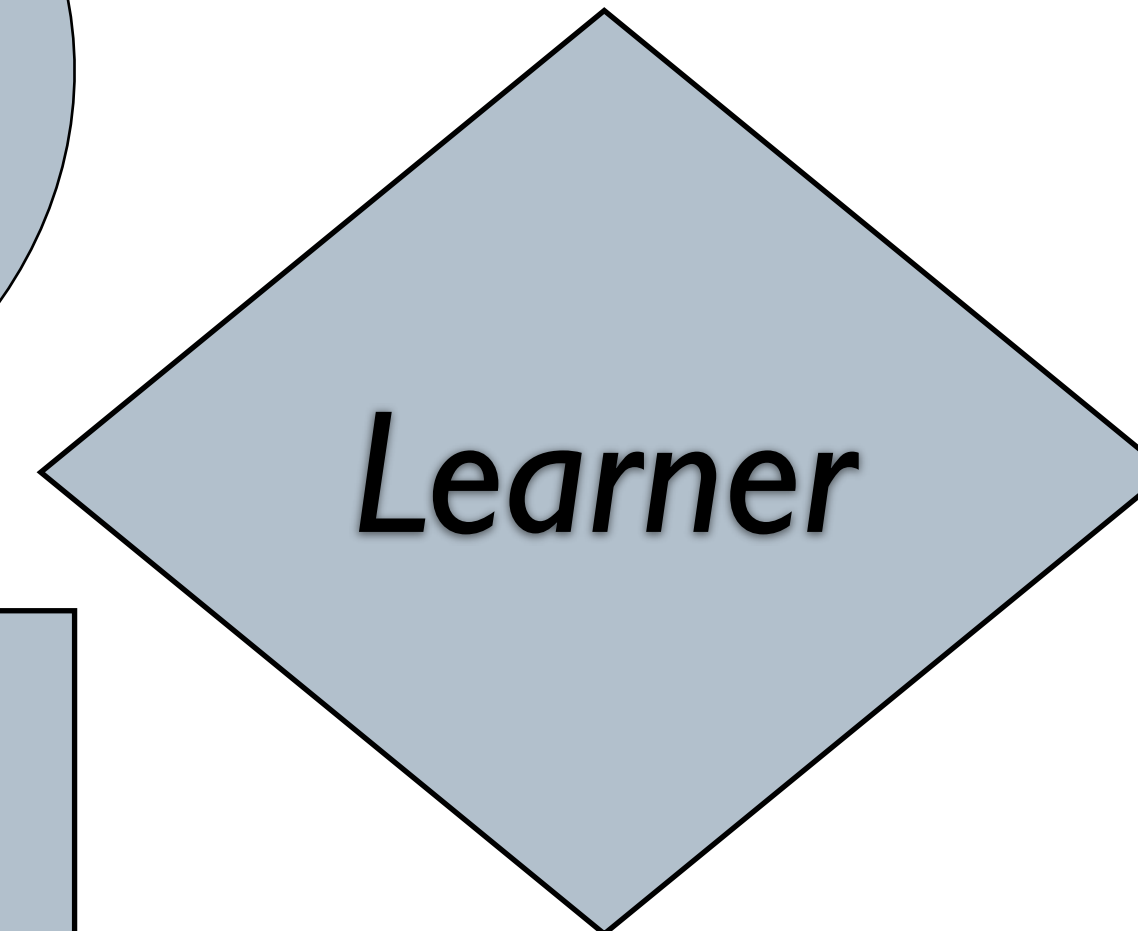
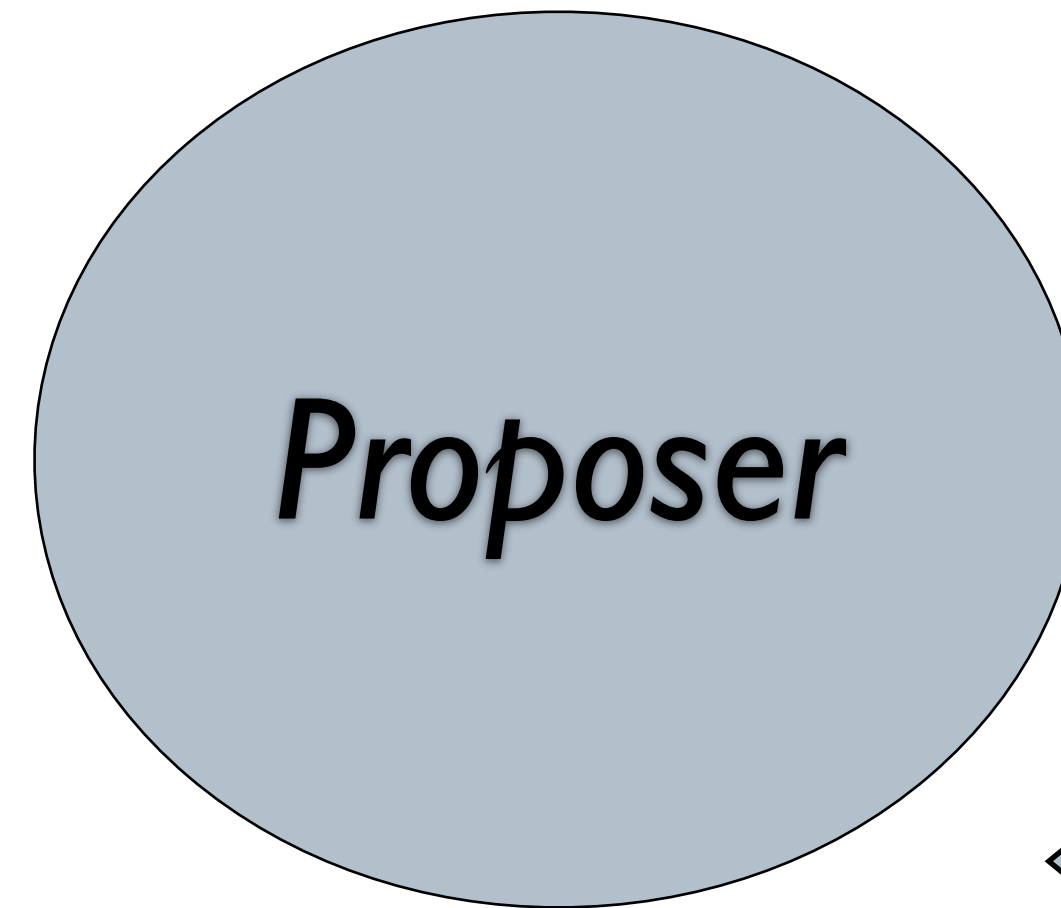
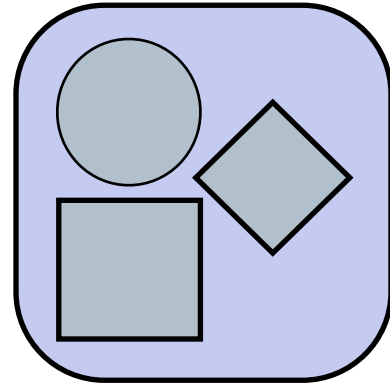


Single-decree Paxos

The Server Replicas



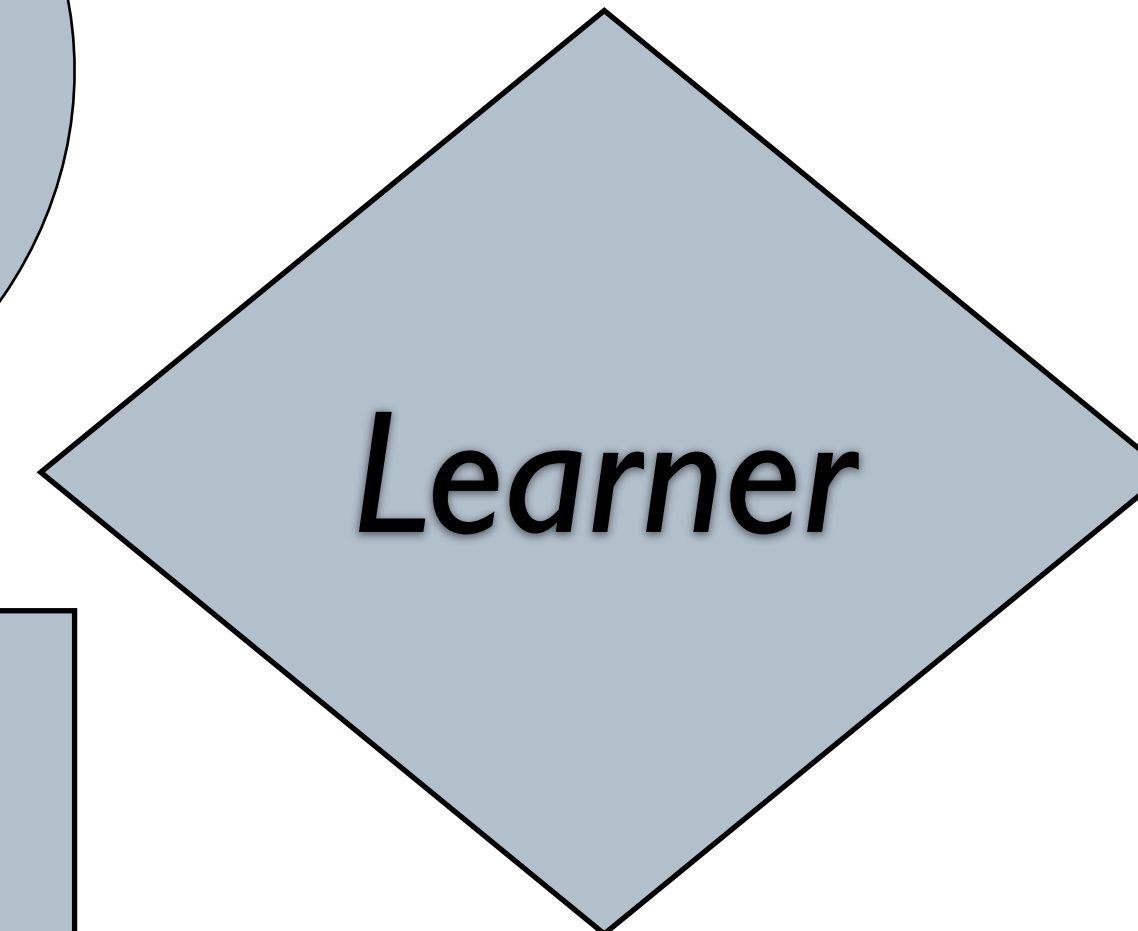
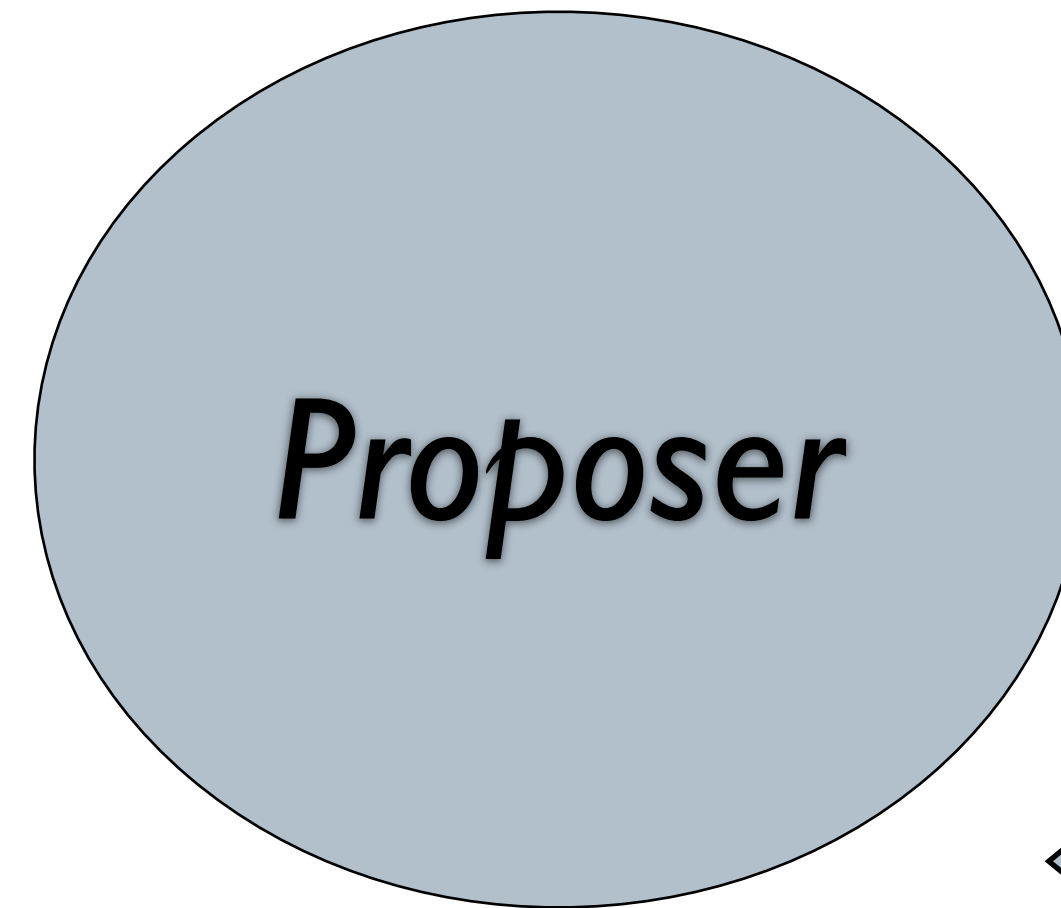
Paxos Agents



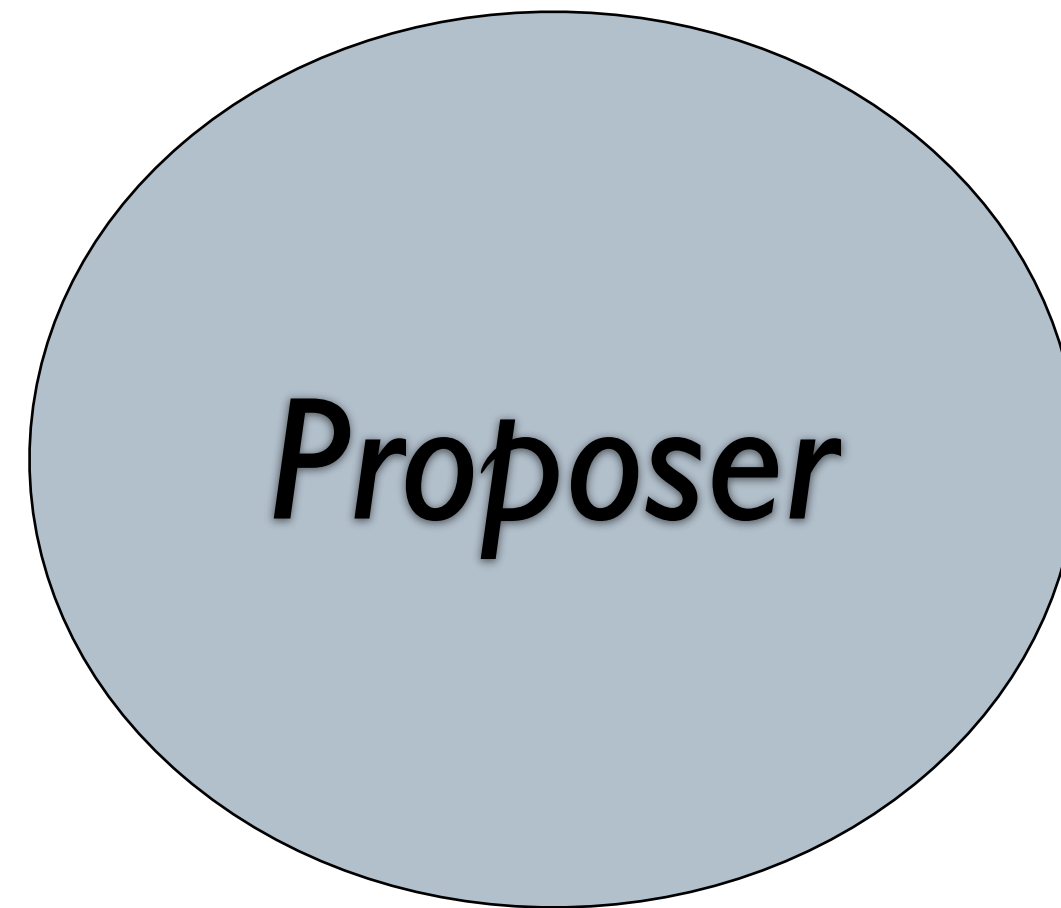
Paxos Agents

Receive commands from clients

Preliminarily order commands
(propose value for consensus)



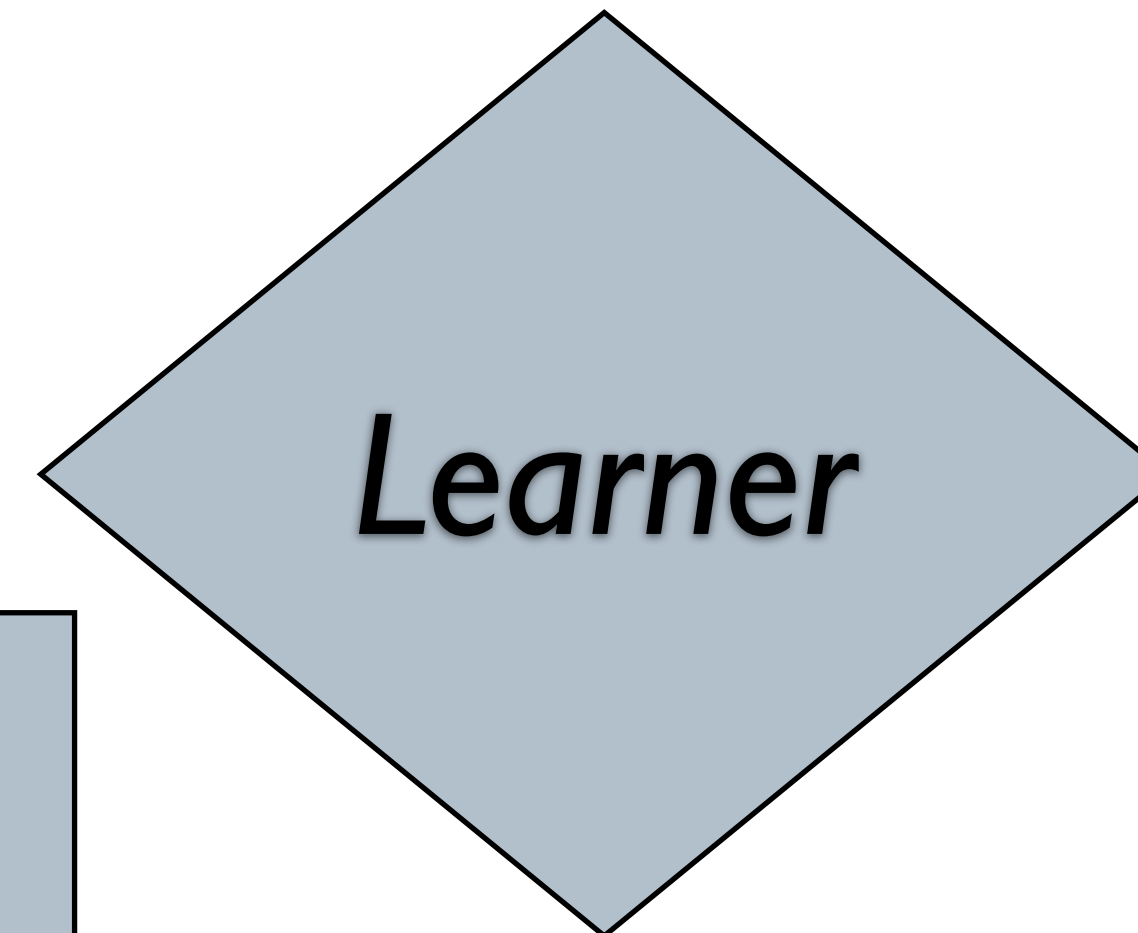
Paxos Agents



Acceptors chooses
the consensus value

Paxos Agents

Learners learn the
consensus value

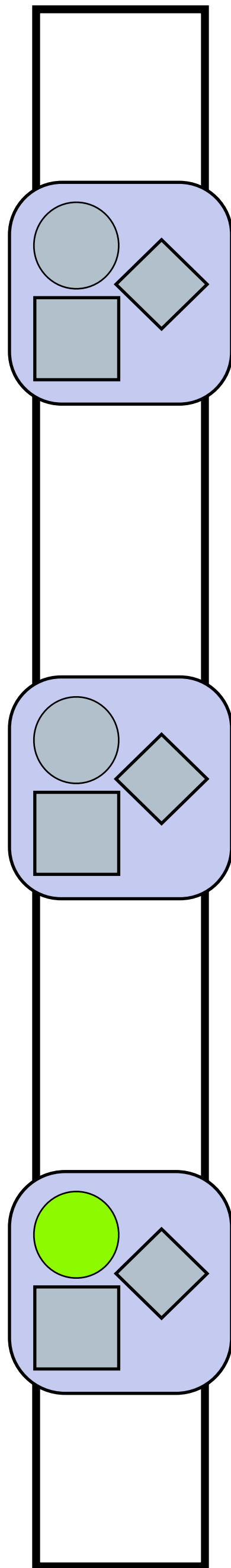


Consensus

- A set of processes tries to choose a common value
- The value represents a client command

Before we move on!

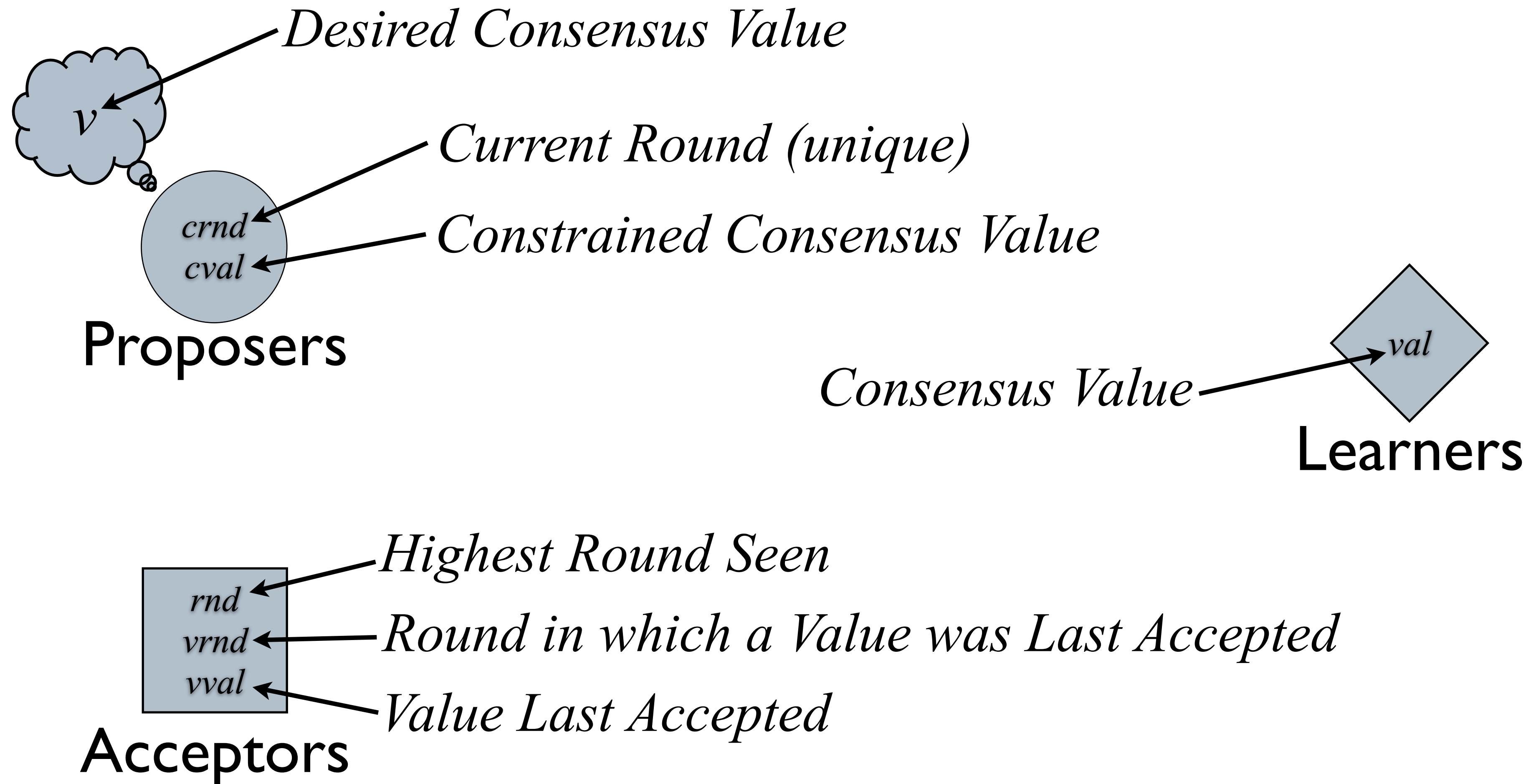
The Proposer



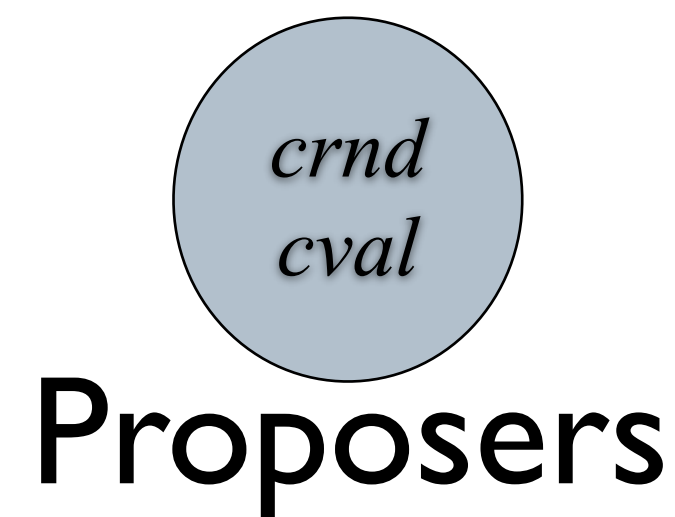
- Common case: there is only one proposer!
- When there is asynchrony we may have
 - Multiple leaders
 - No leader

Leader

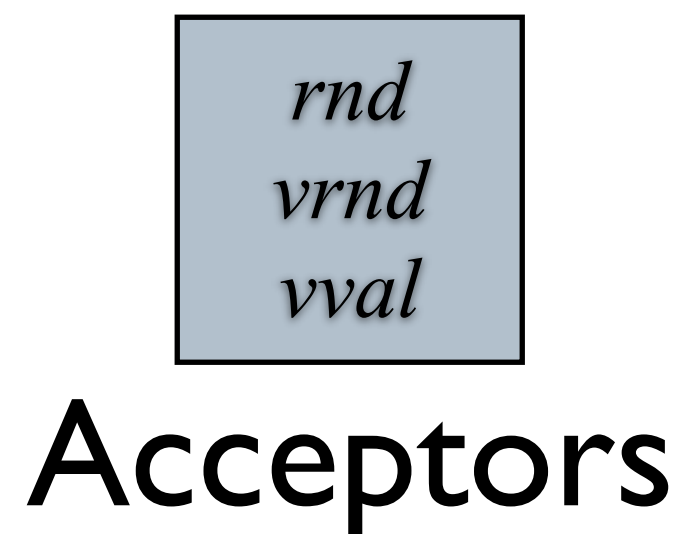
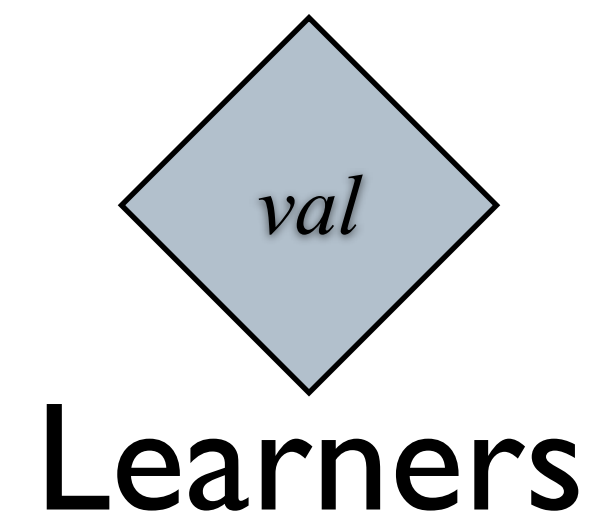
Paxos Agents



Paxos Agents - The API



prepare(crnd)
accept(crnd,cval)



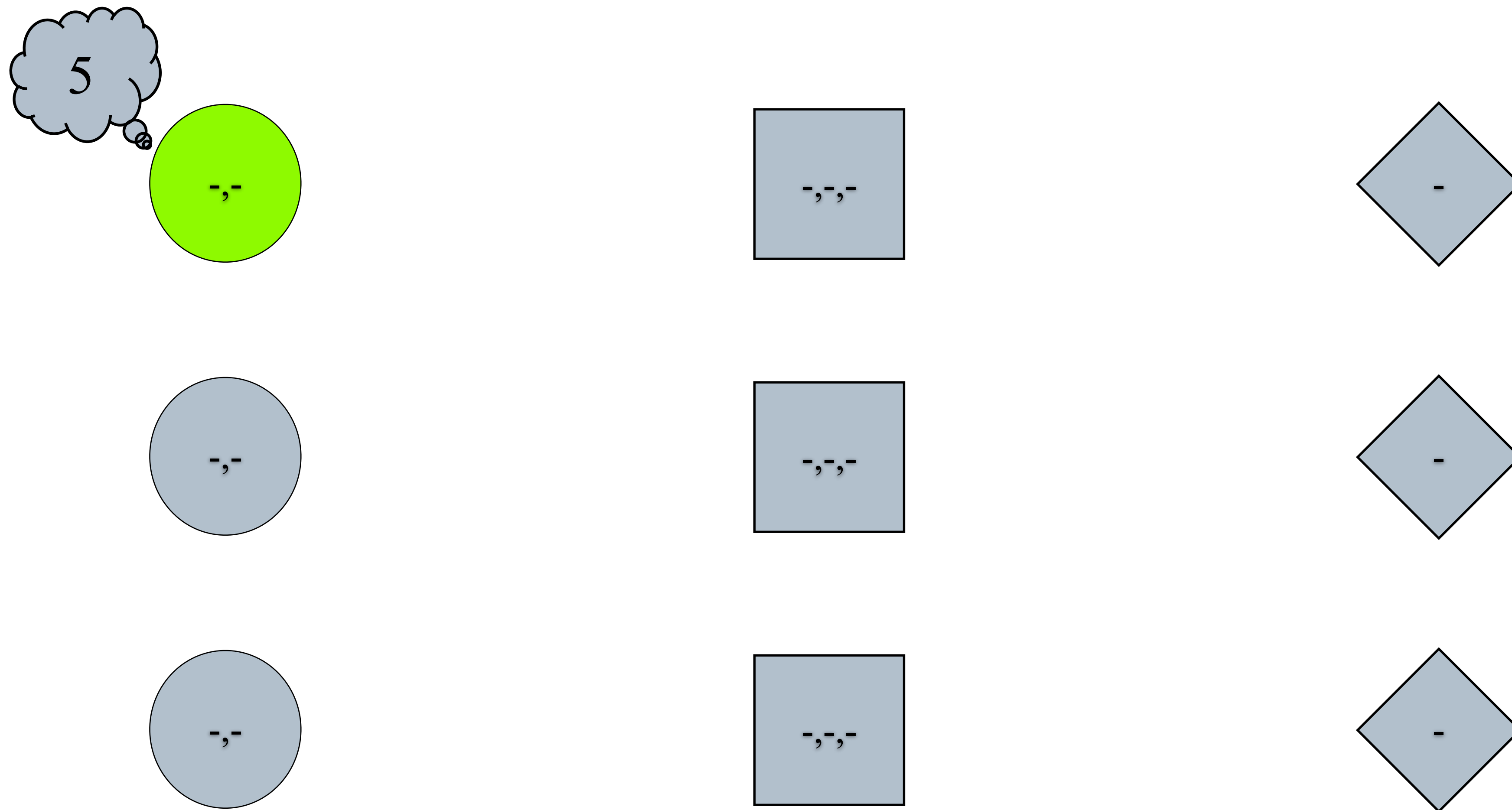
promise(rnd, vrnd, vval)
learn(rnd, vval)

Paxos Examples

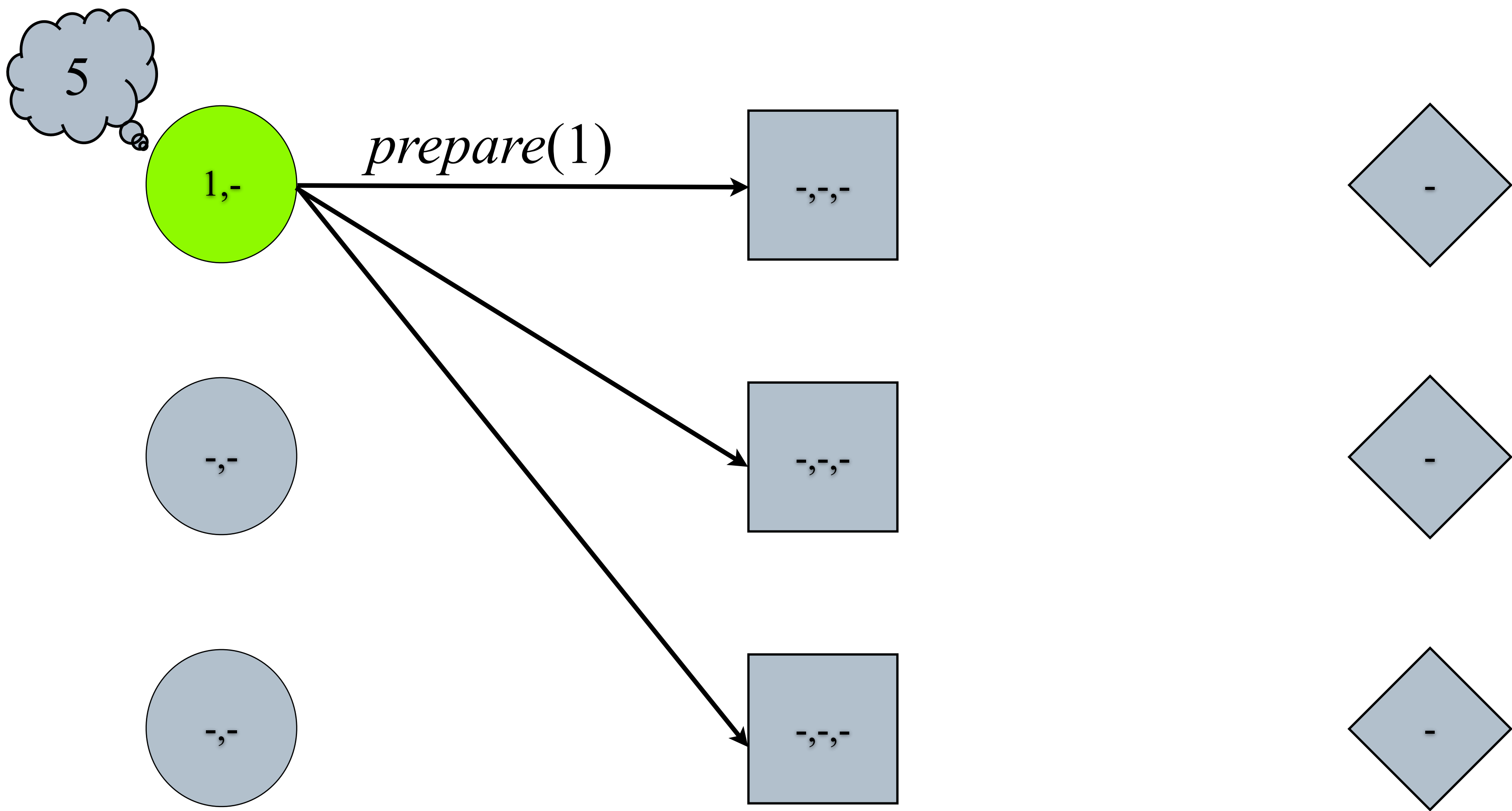
Example I

A Full Paxos Execution

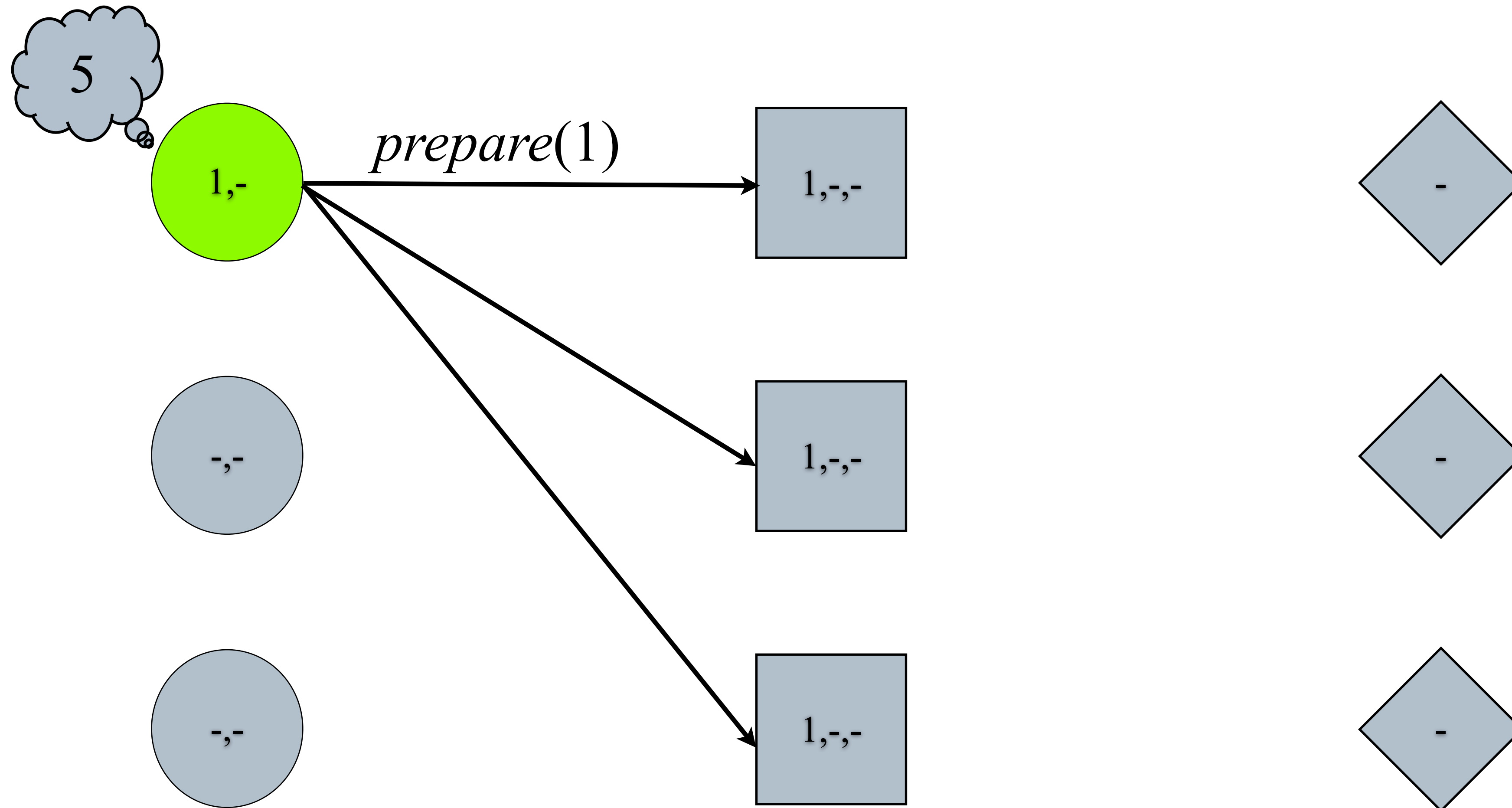
Full Paxos Execution



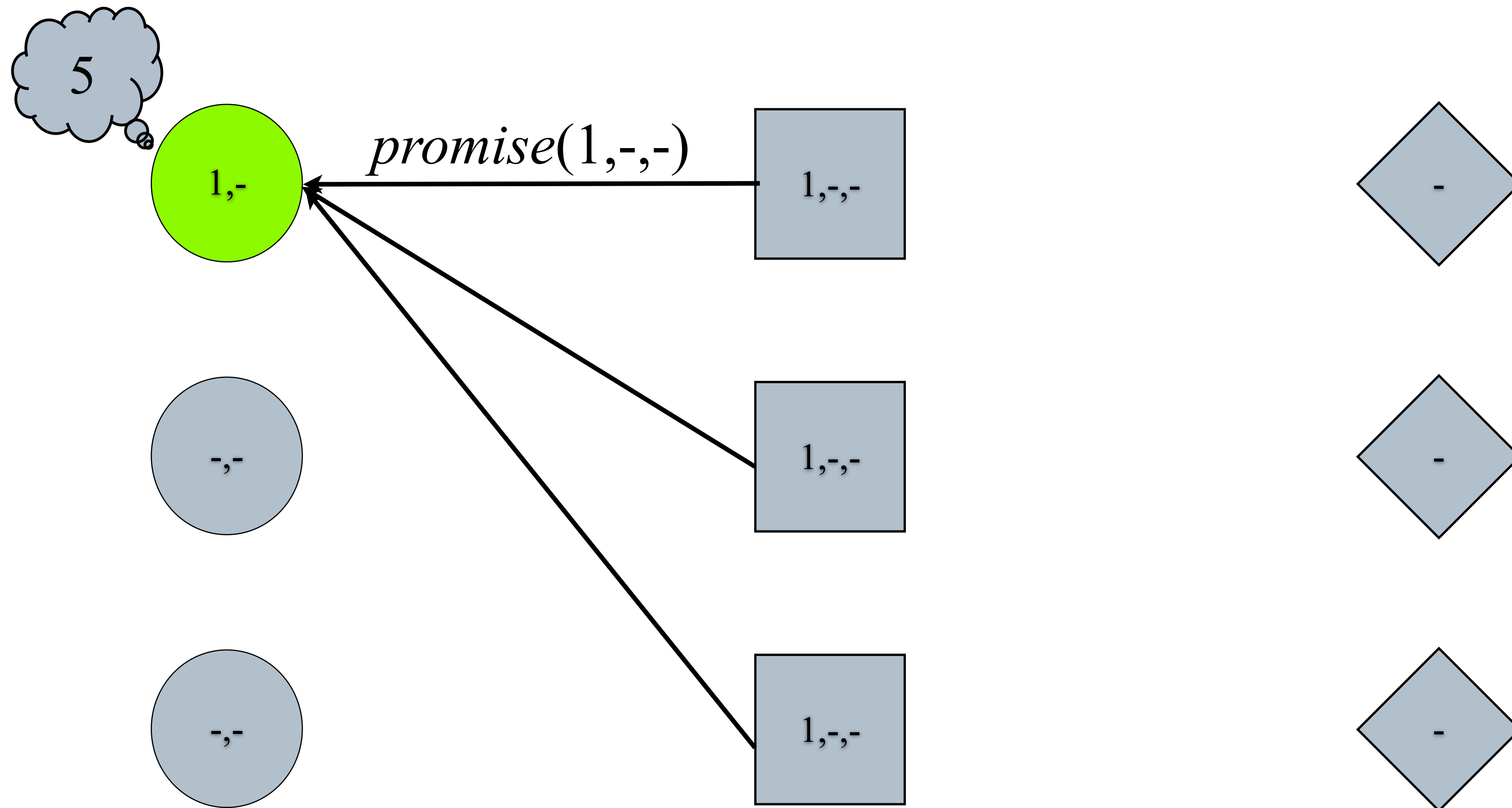
Full Paxos Execution



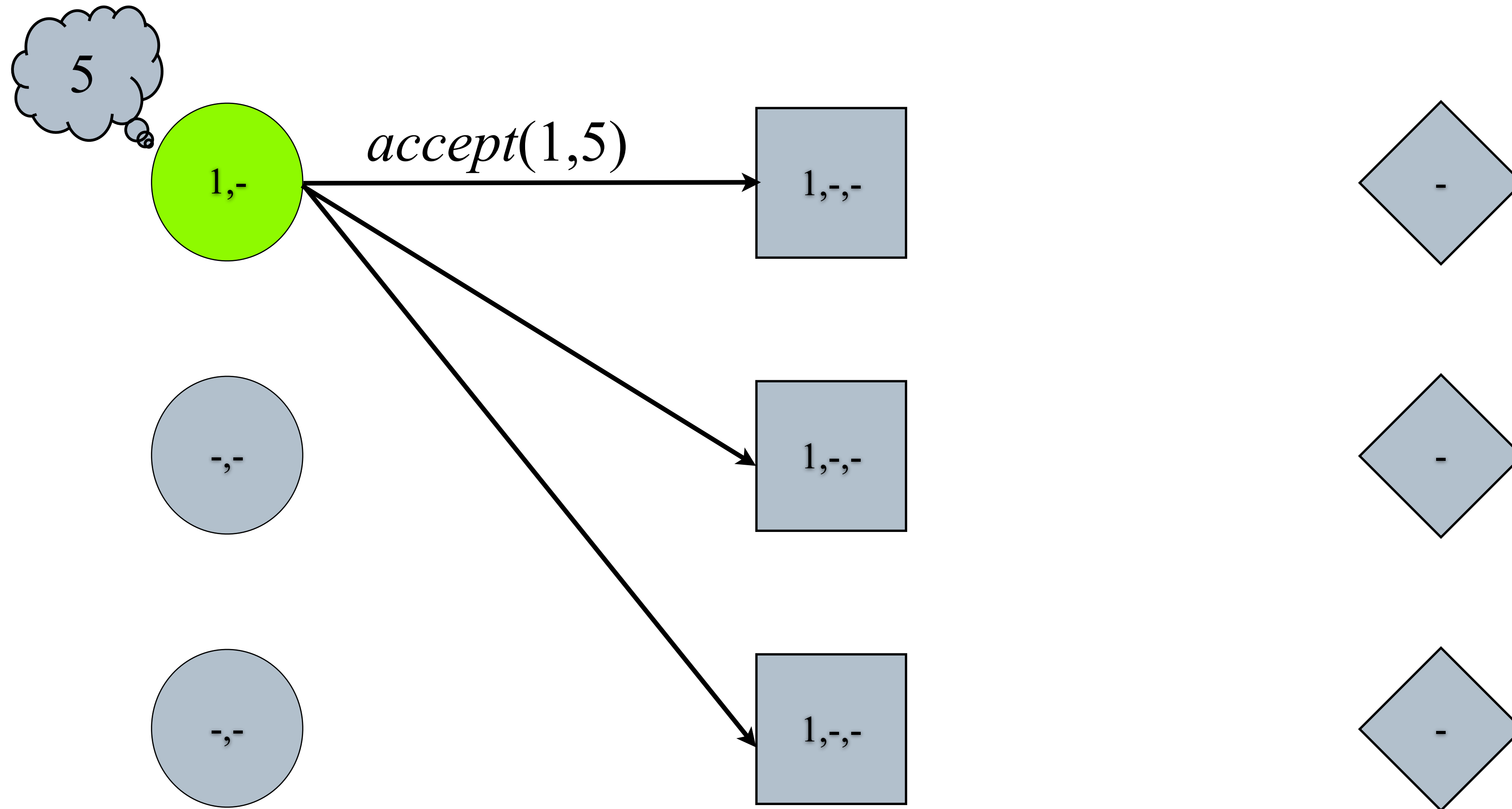
Full Paxos Execution



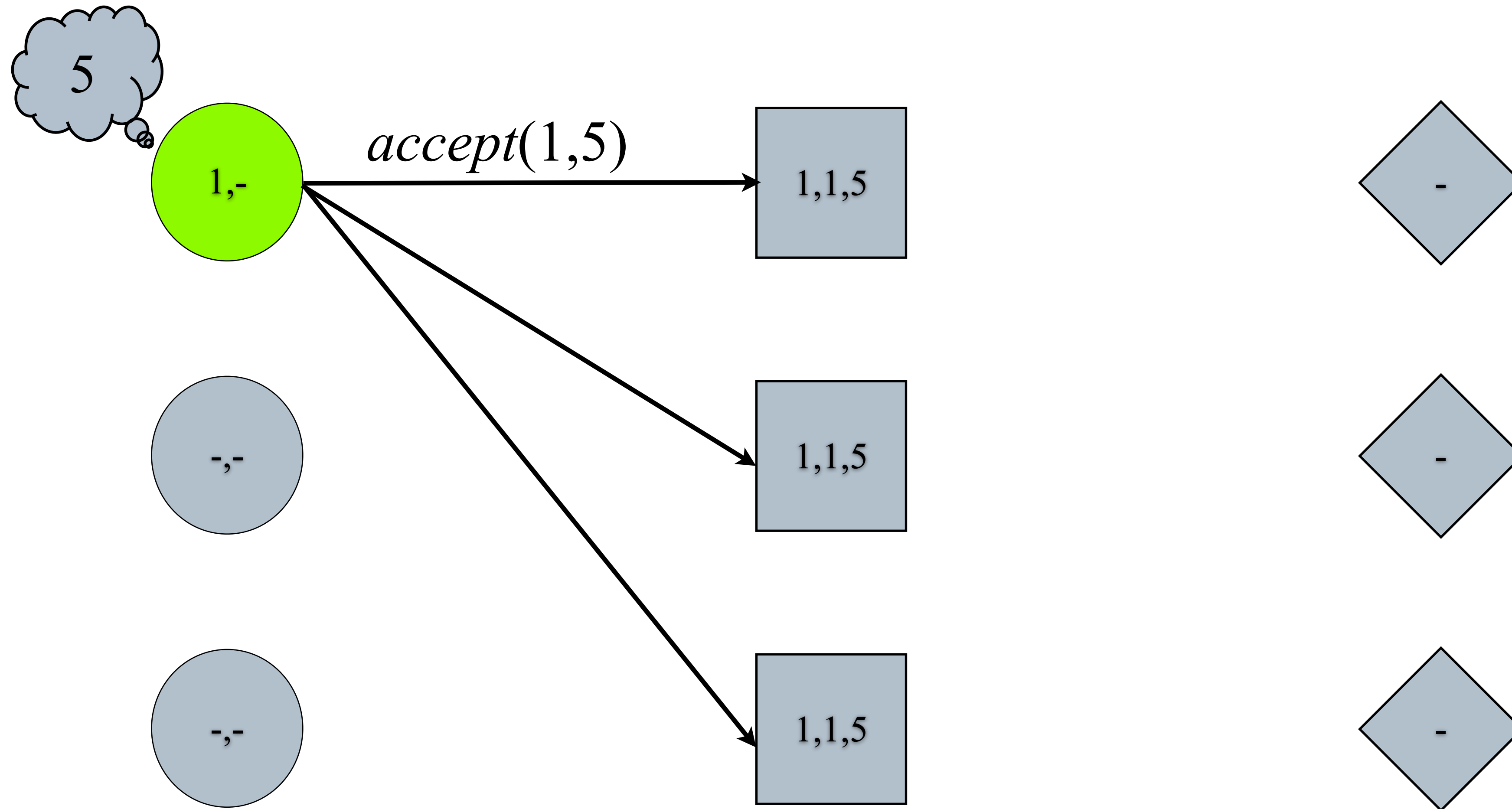
Full Paxos Execution



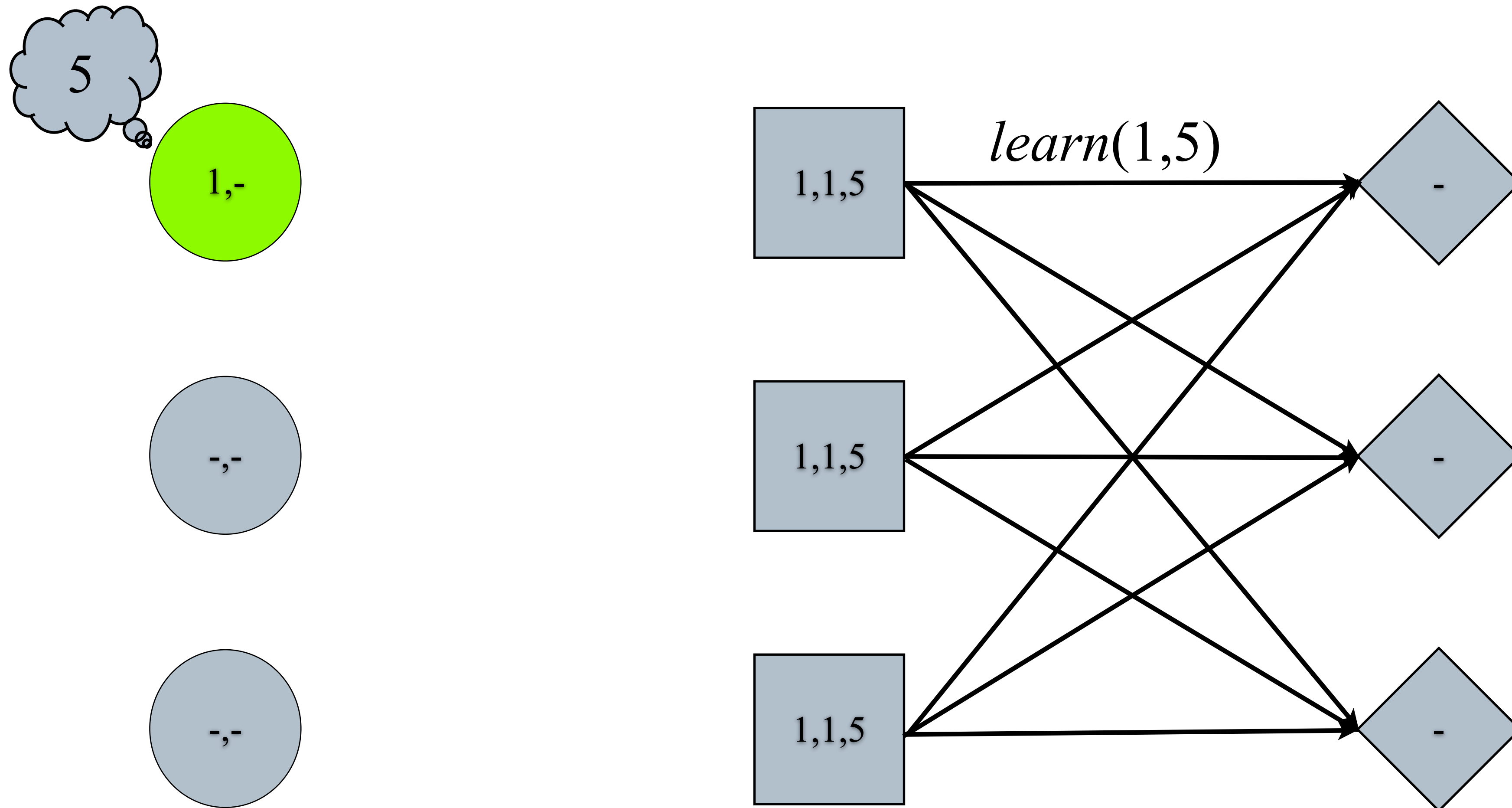
Full Paxos Execution



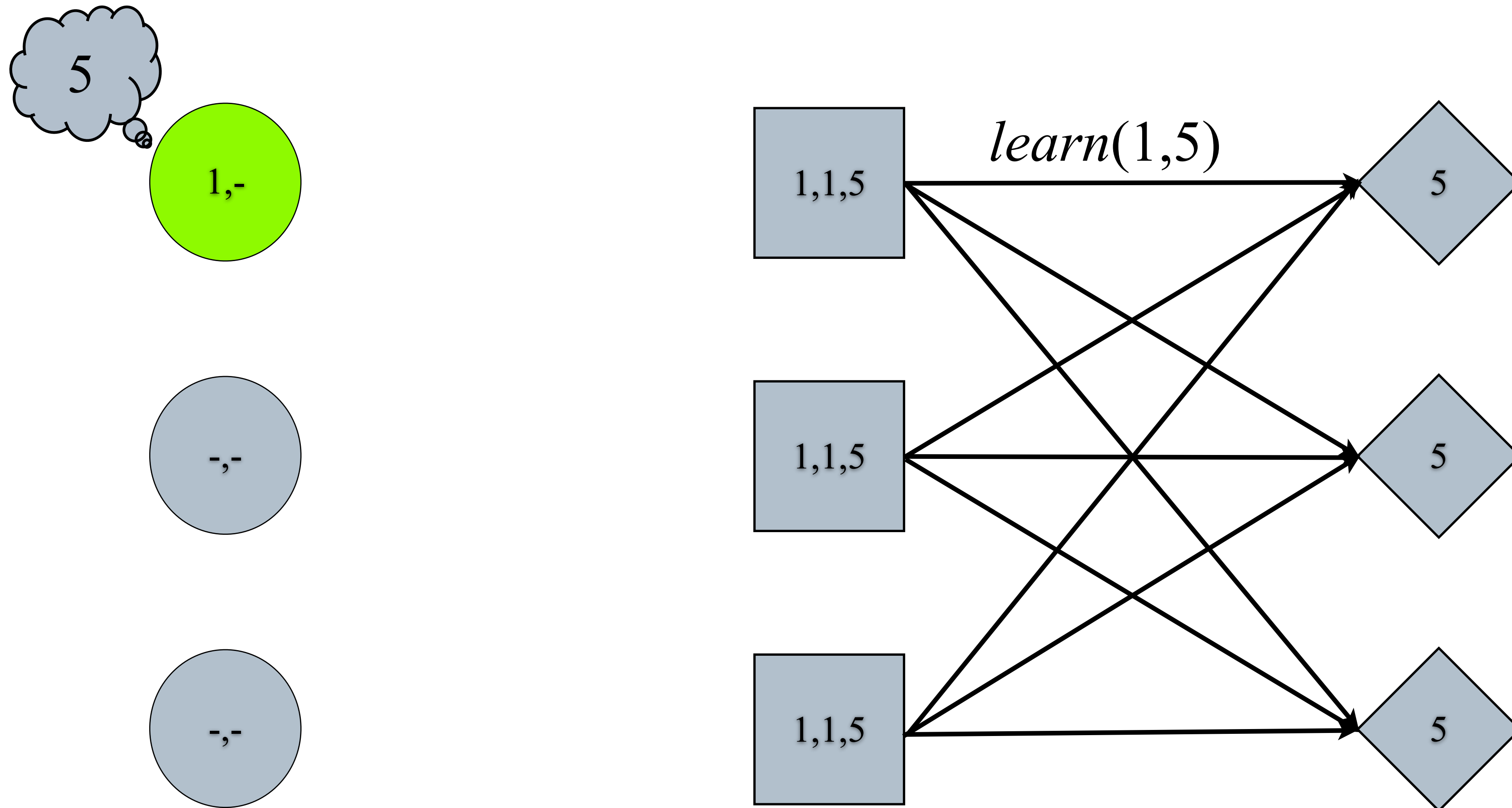
Full Paxos Execution



Full Paxos Execution



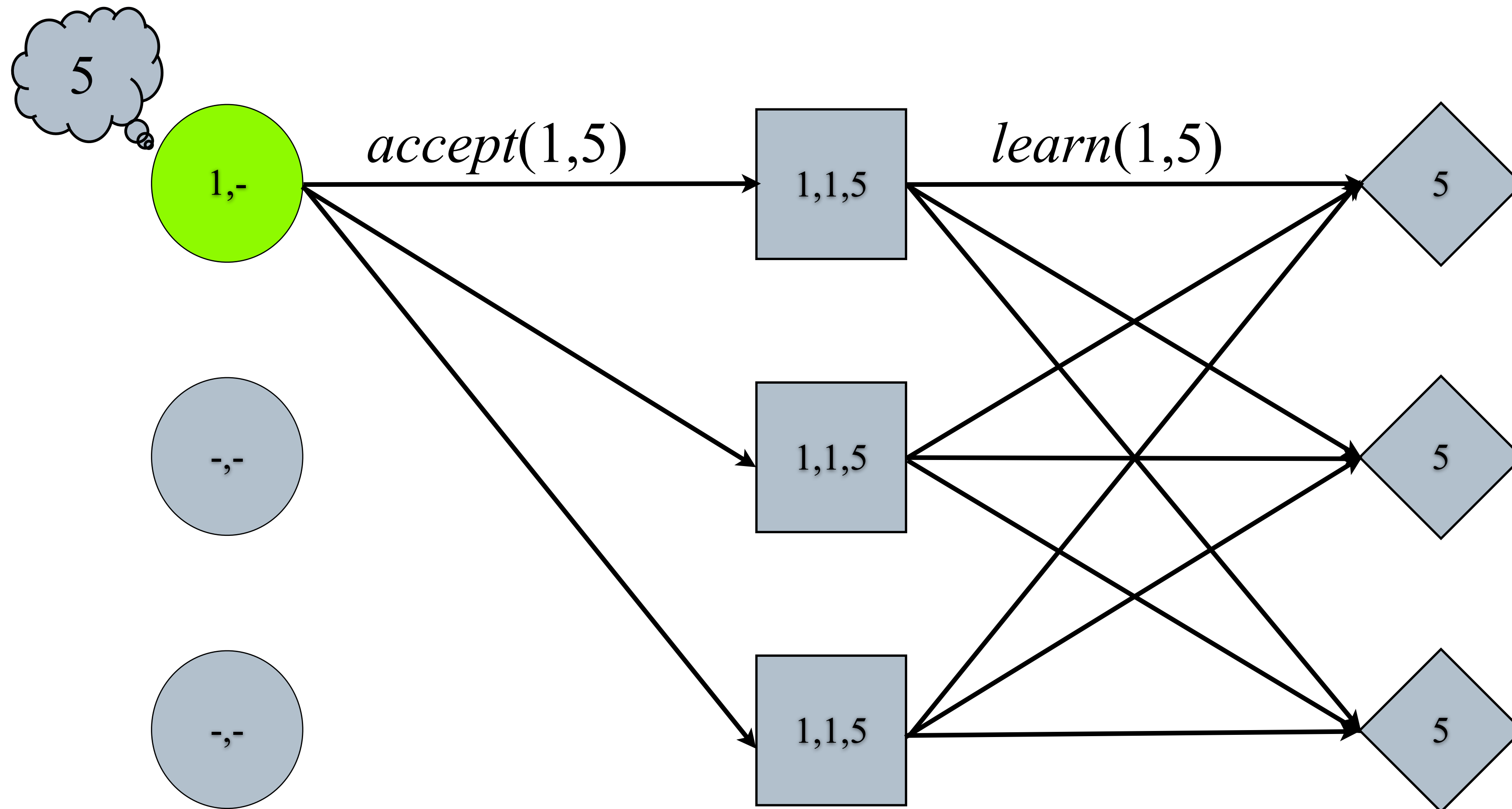
Full Paxos Execution



Paxos Optimization

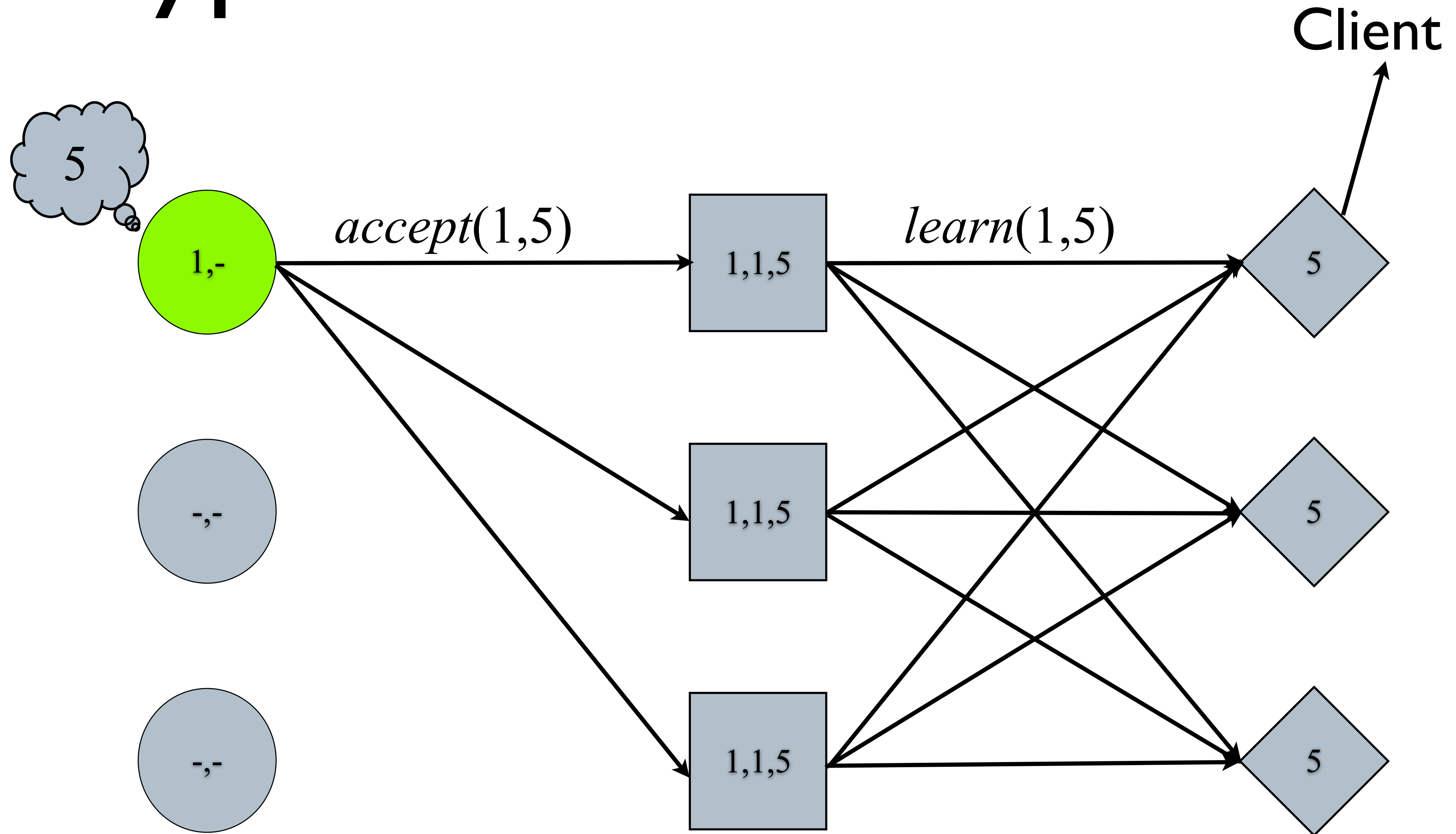
- Leader is stable across multiple Slots
 - Skip the two first message exchanges
- Does not work if multiple proposers think they are leader
 - This may cause multiple rounds of msg exchanges

Typical Paxos Execution



Phase 2

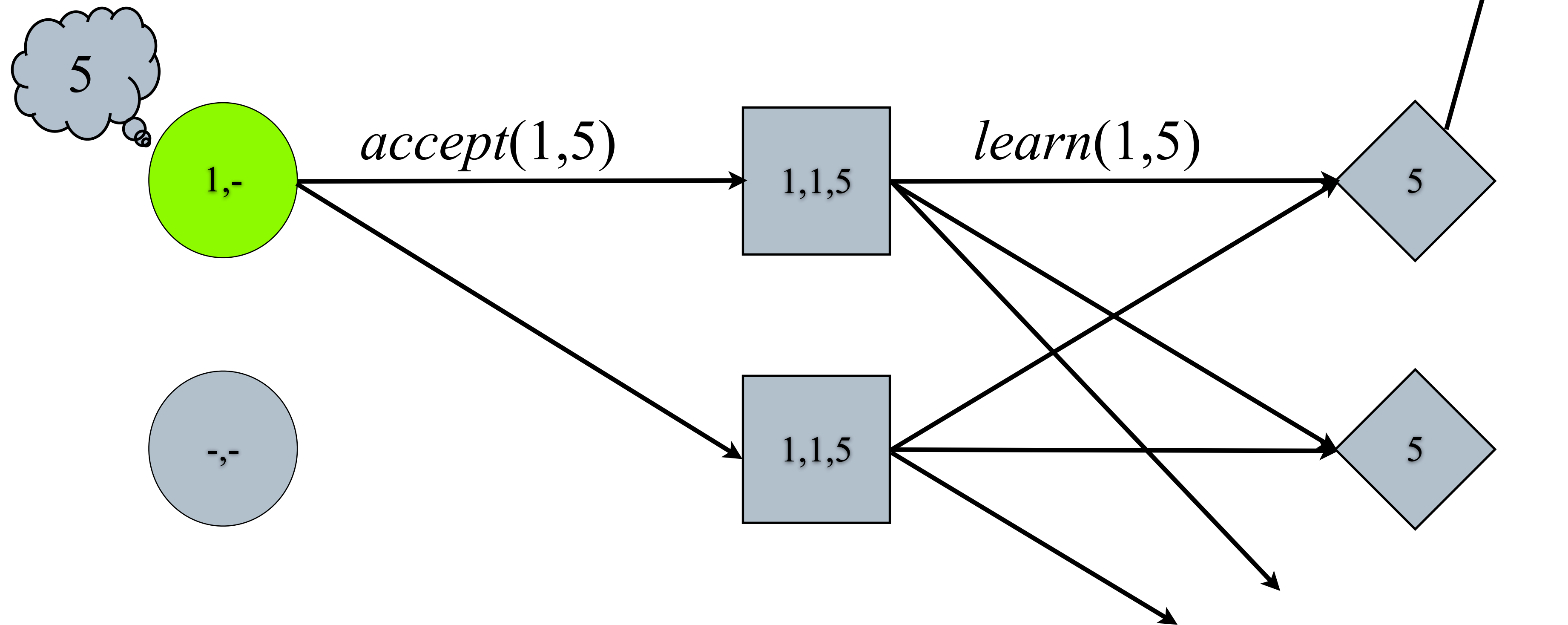
Typical Paxos Execution



Paxos Execution

With a Failed Replica

Phase 2



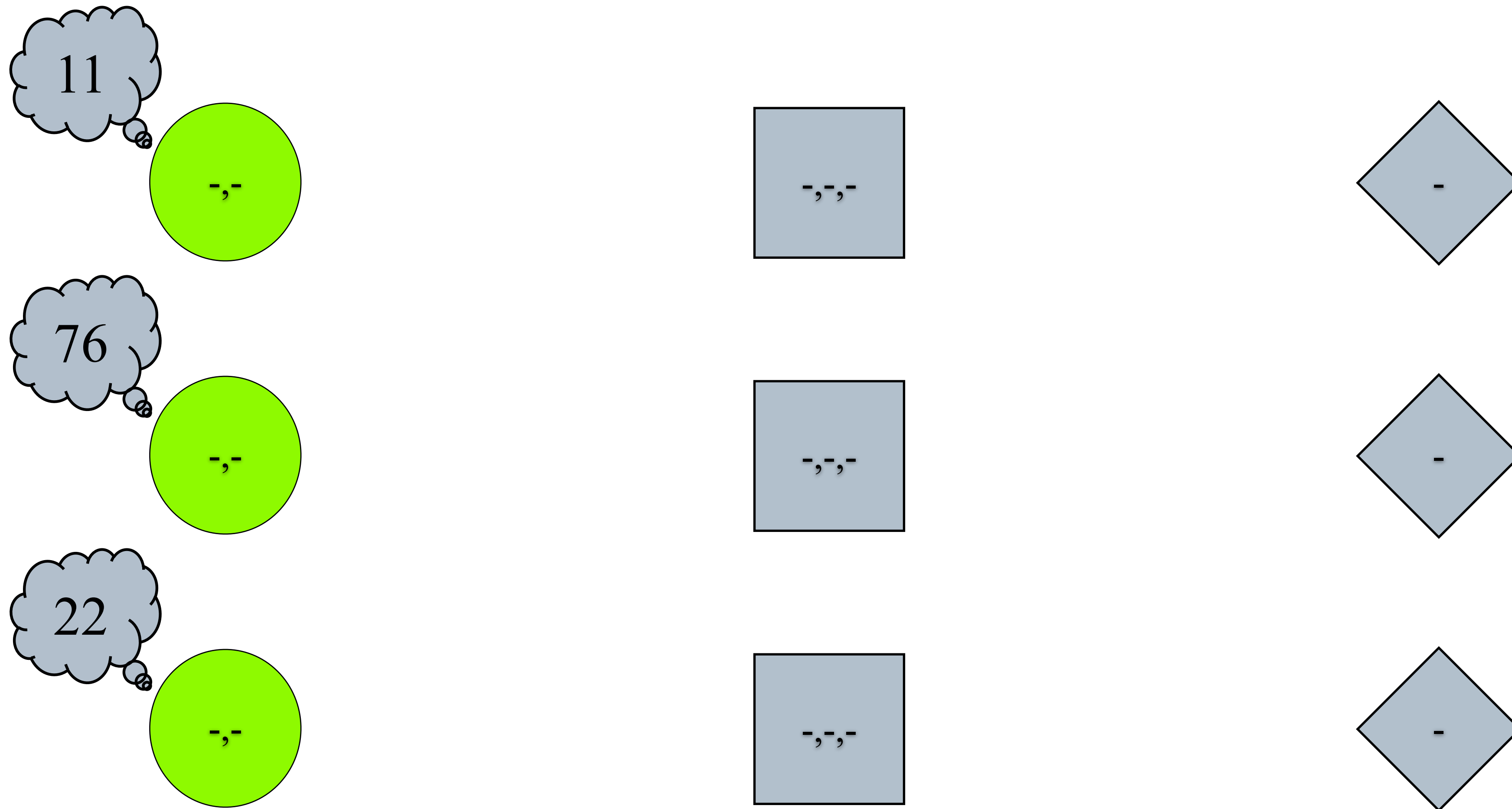
Example II

Problematic Paxos

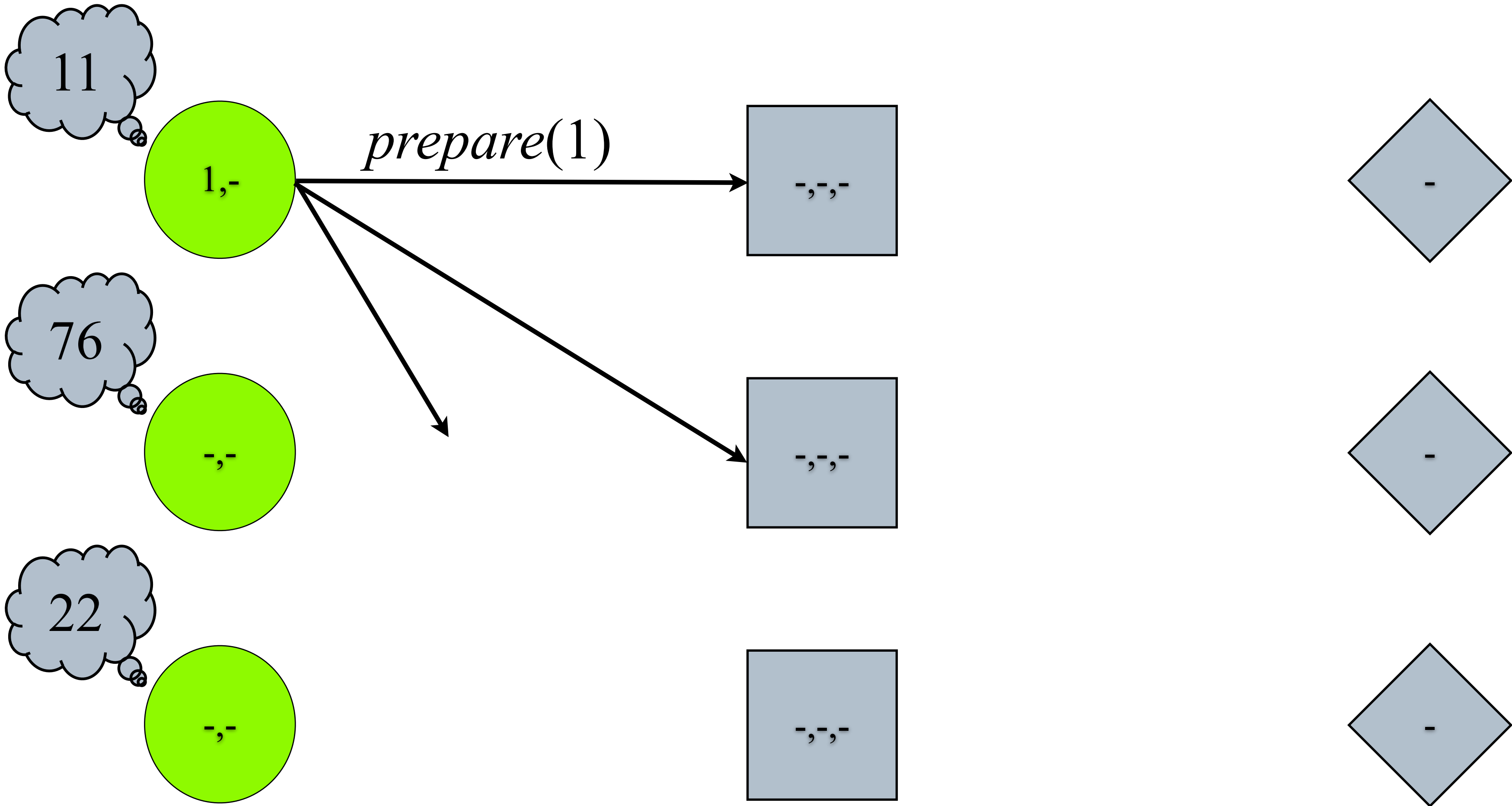
Execution:

Concurrent Leaders

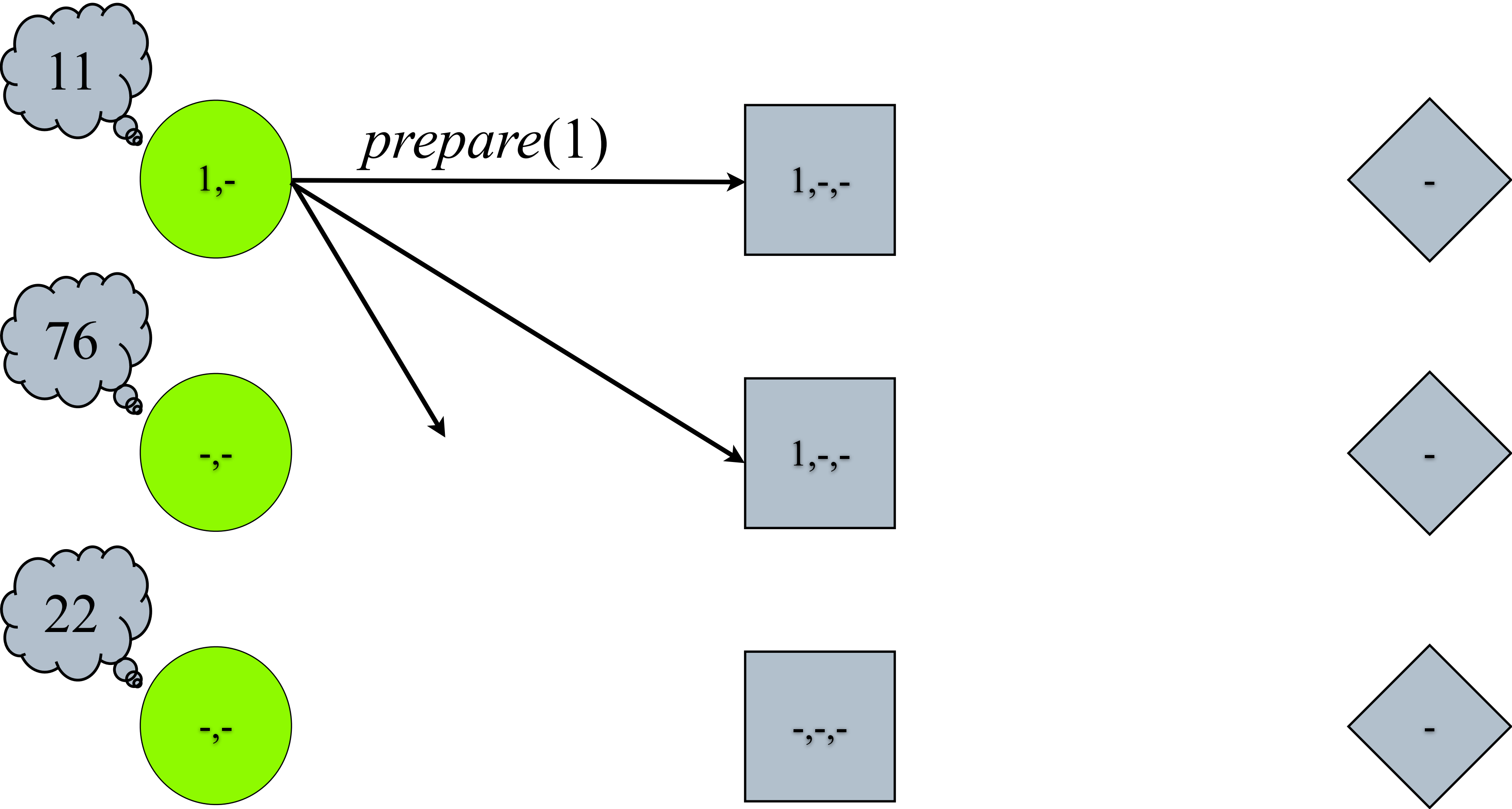
Concurrent Leaders



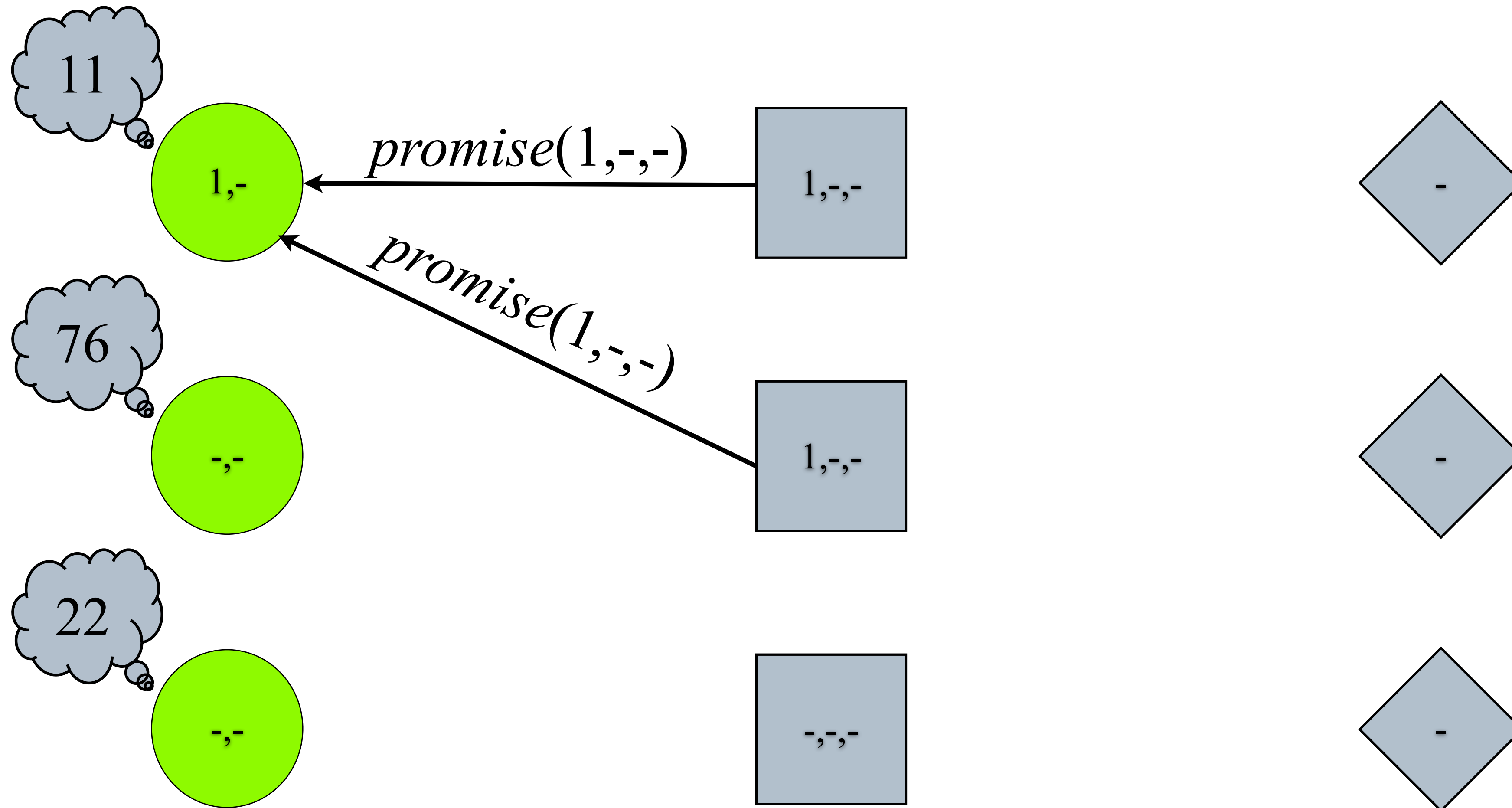
Concurrent Leaders



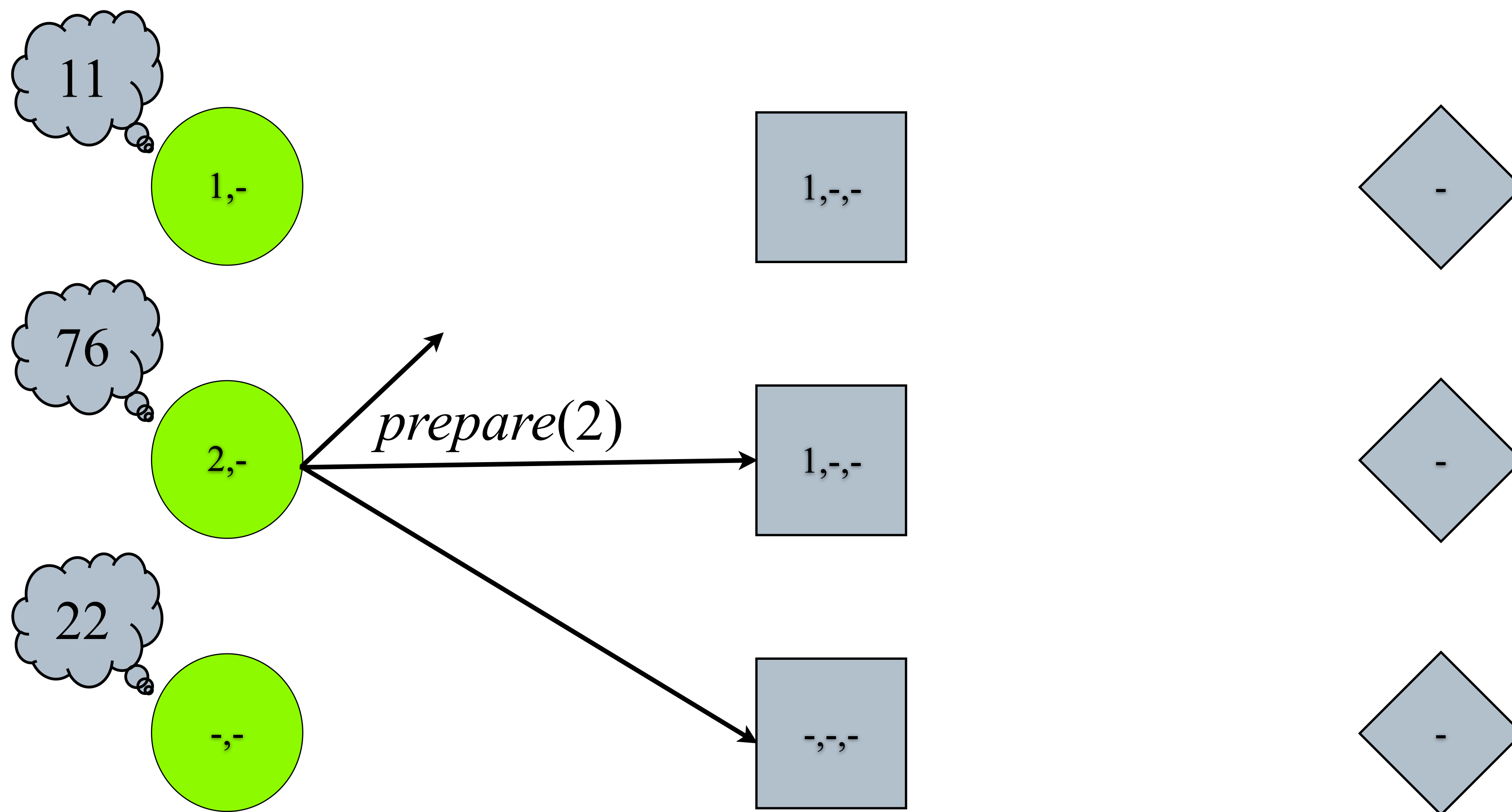
Concurrent Leaders



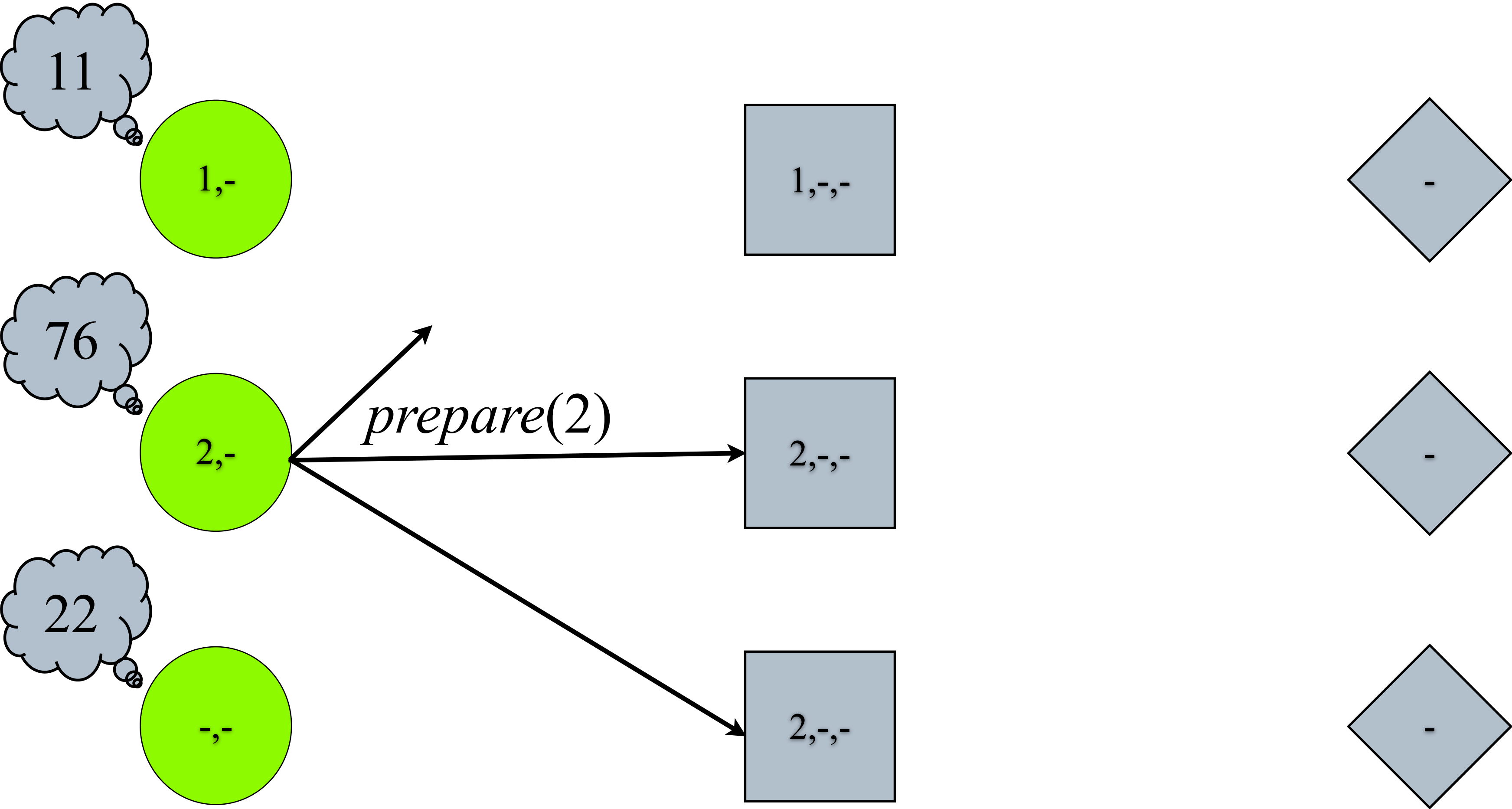
Concurrent Leaders



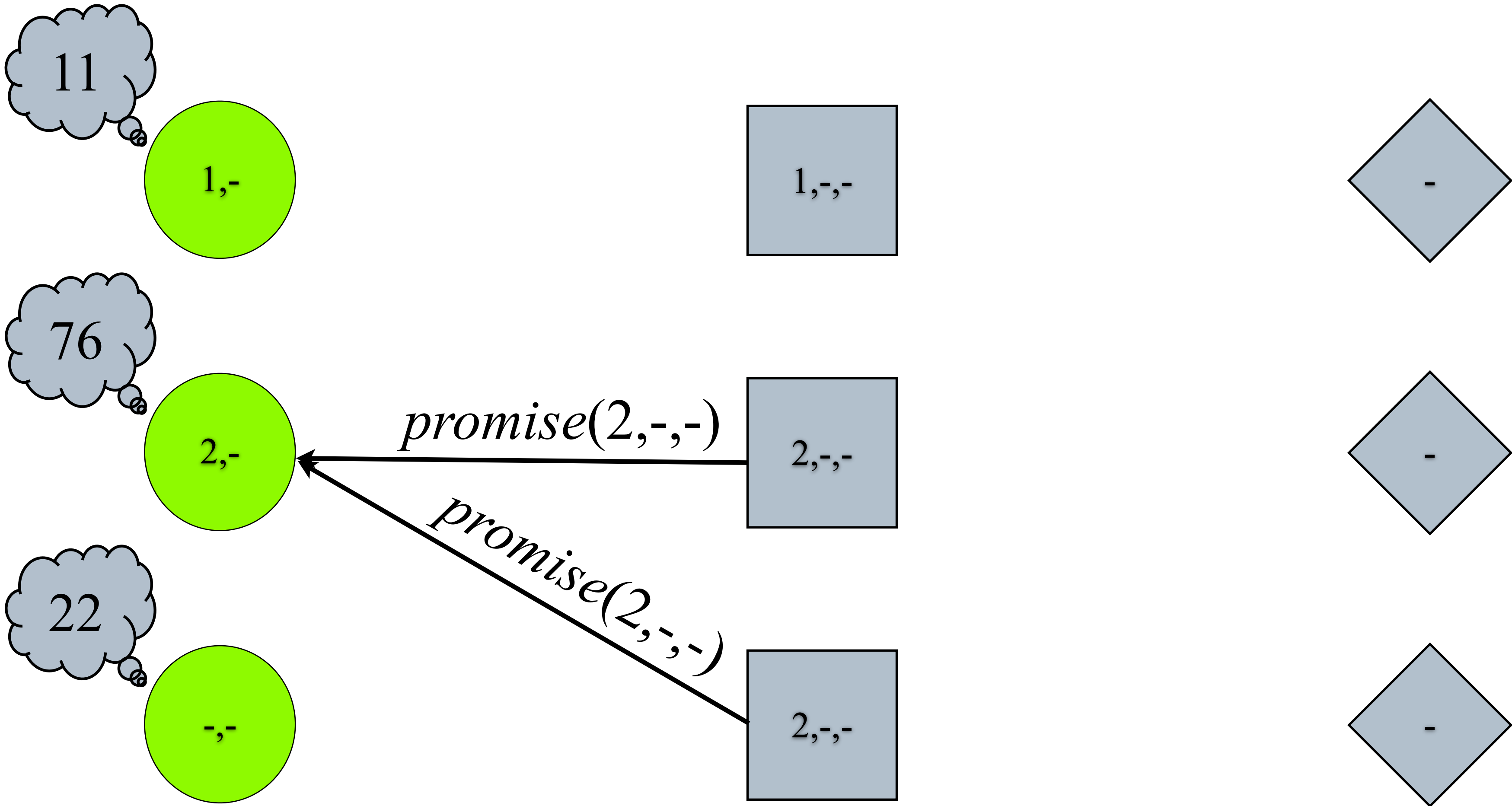
Concurrent Leaders



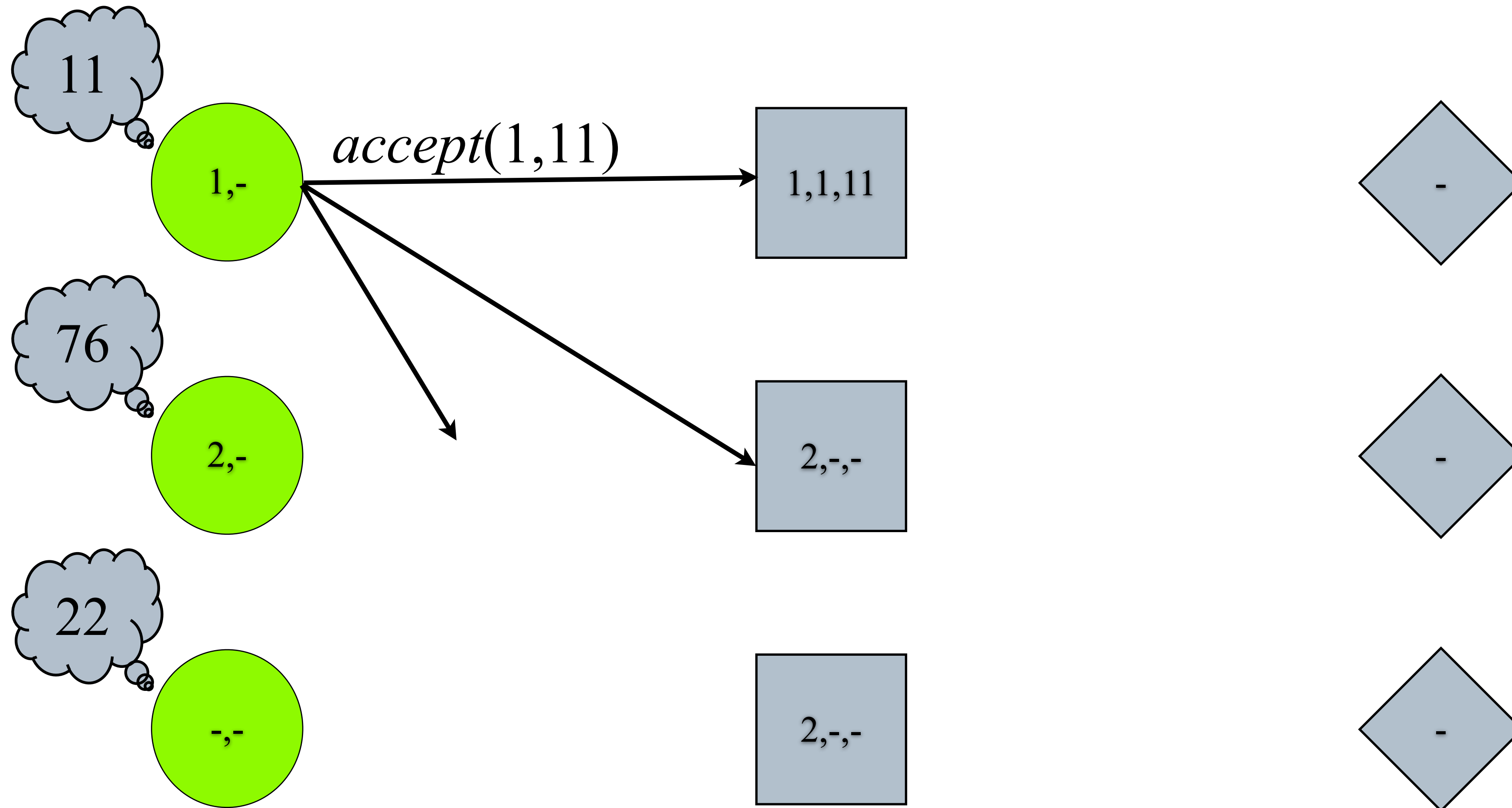
Concurrent Leaders



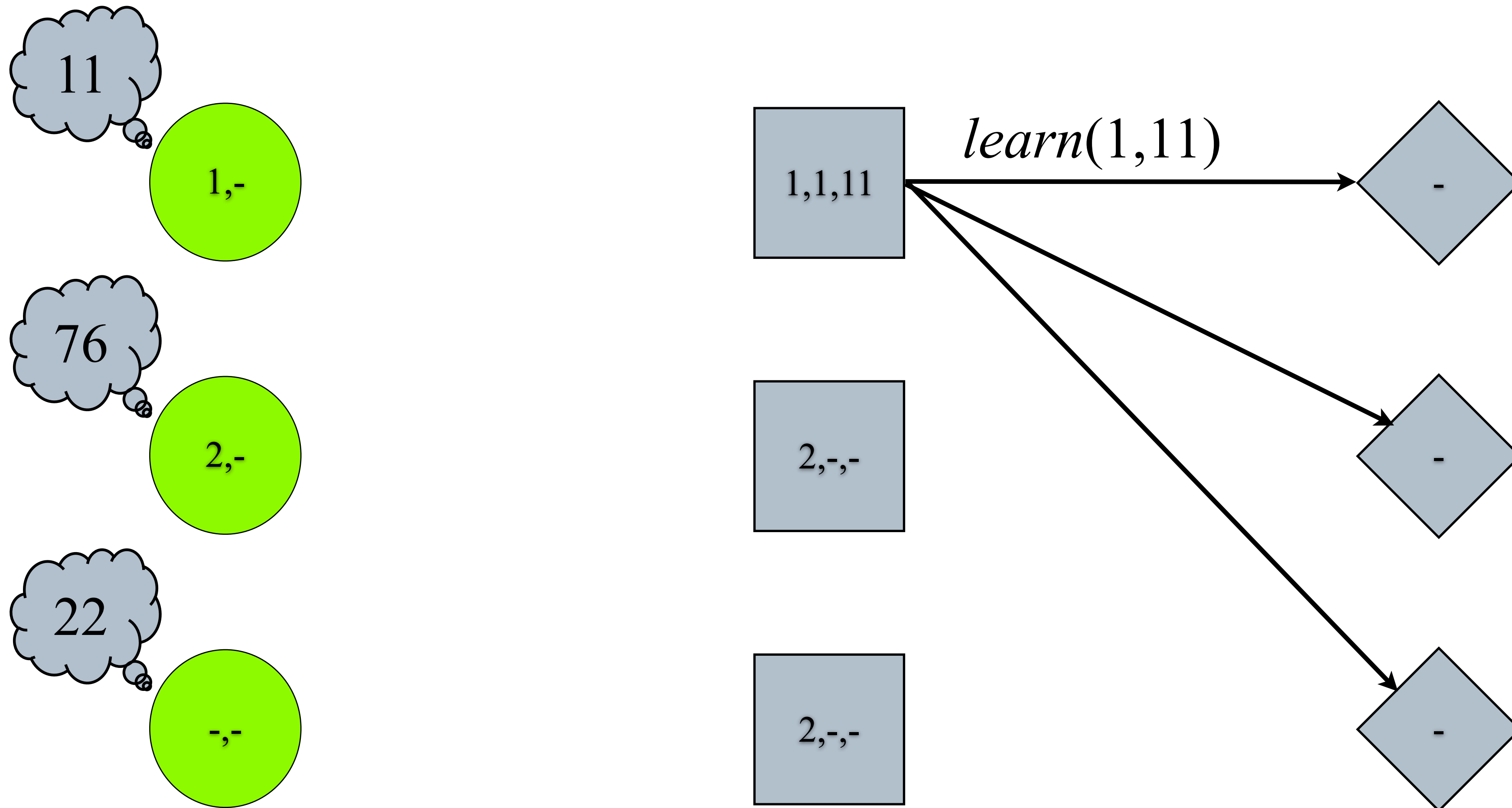
Concurrent Leaders



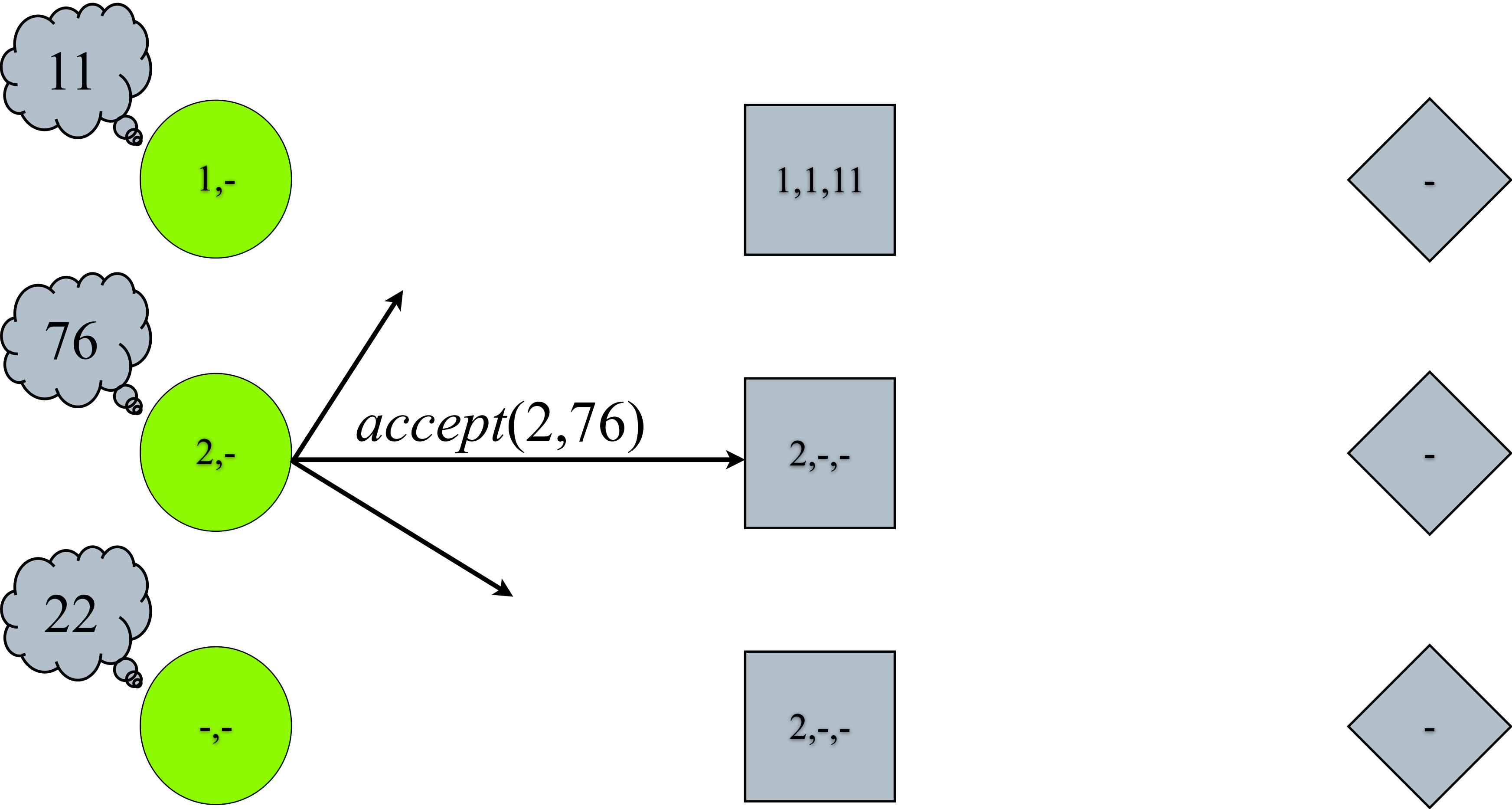
Concurrent Leaders



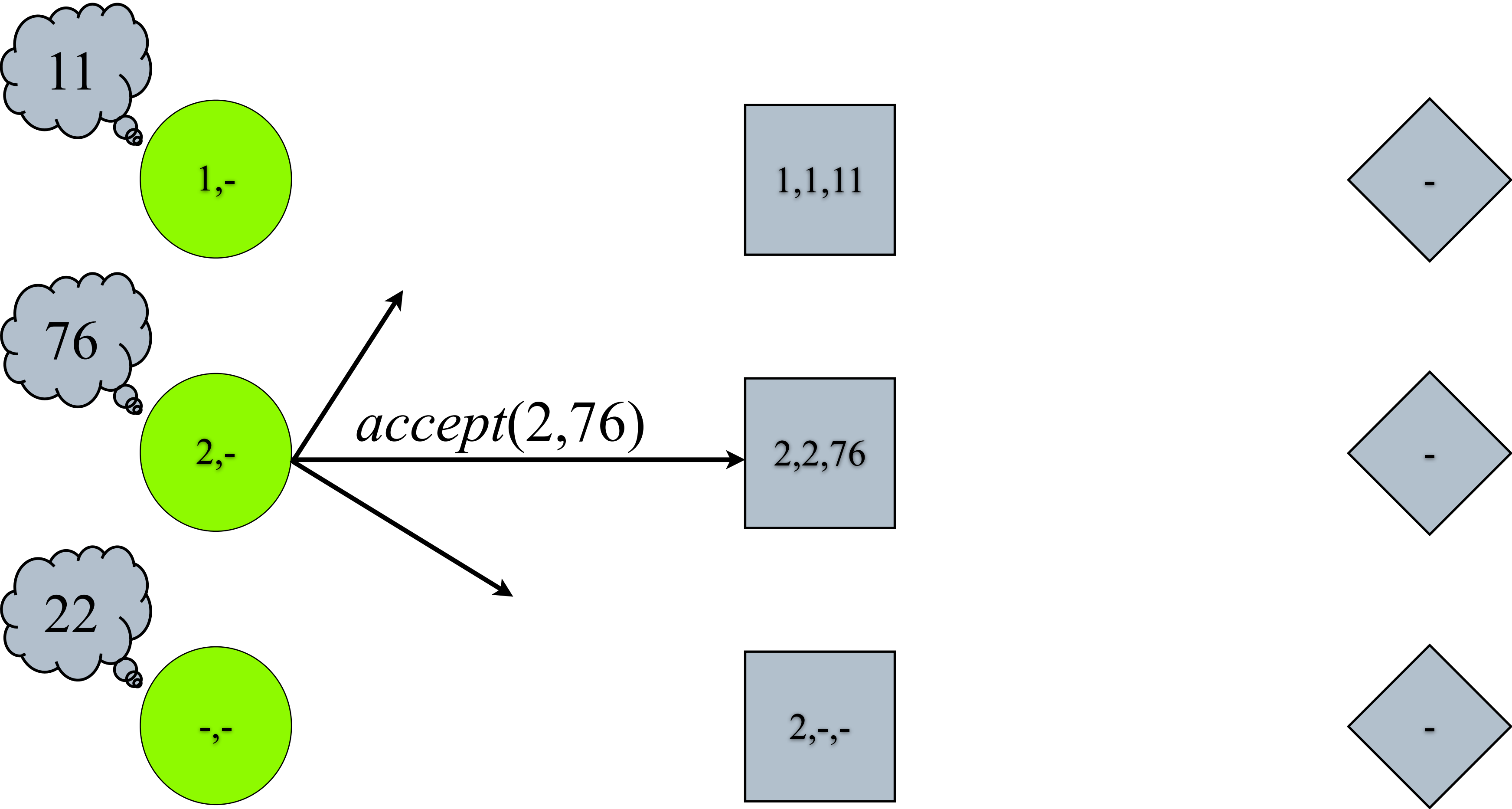
Concurrent Leaders



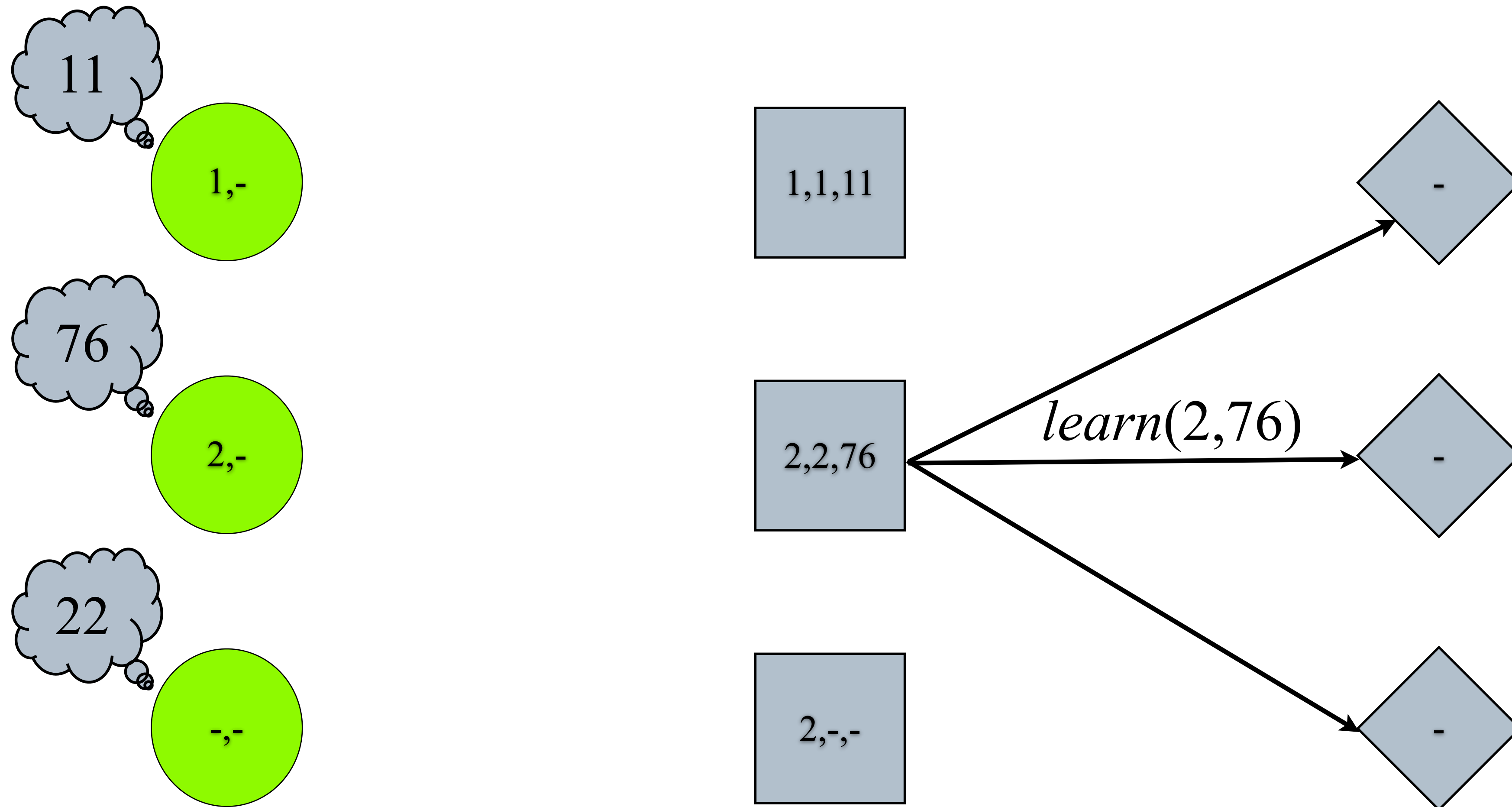
Concurrent Leaders



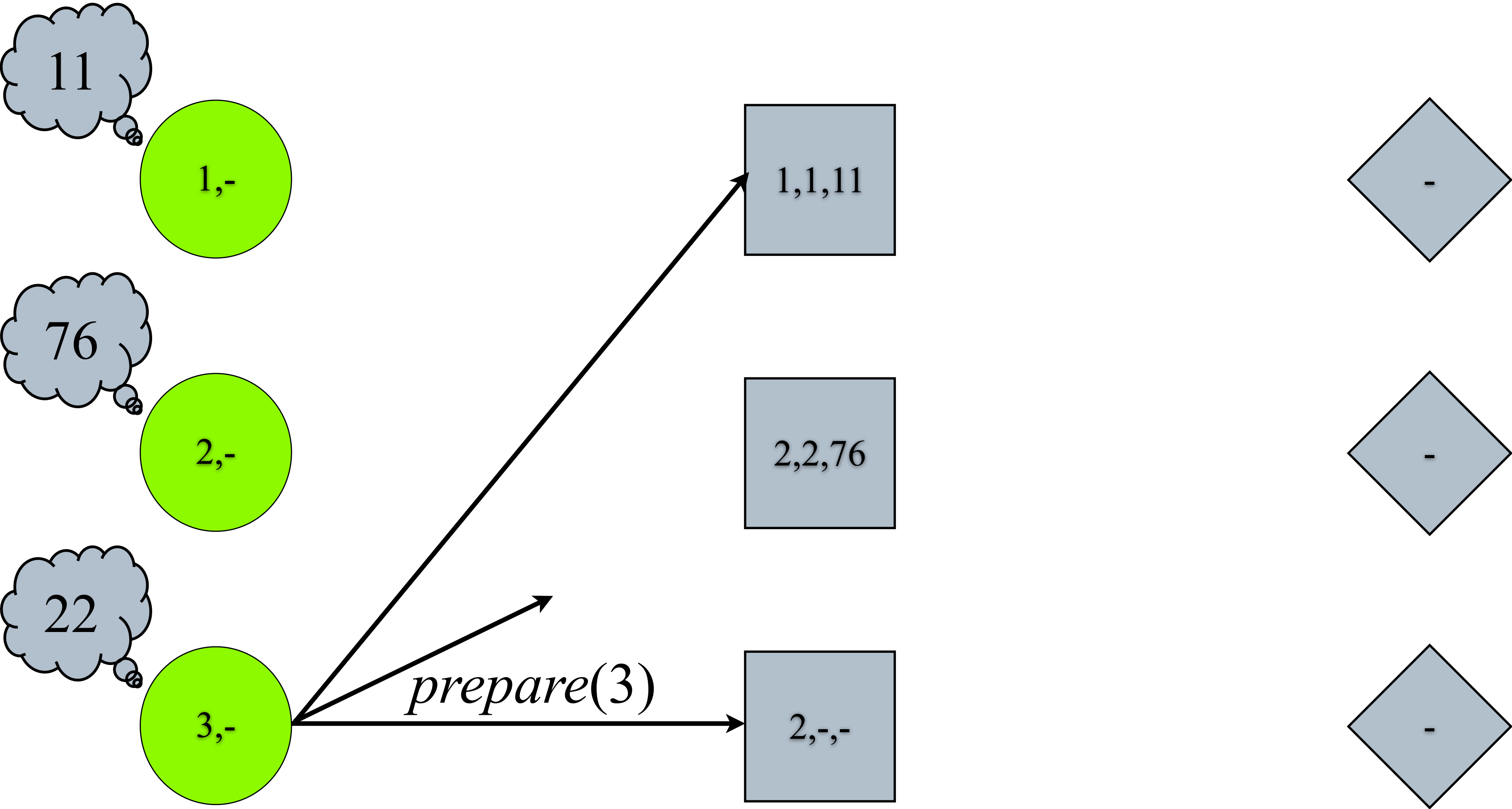
Concurrent Leaders



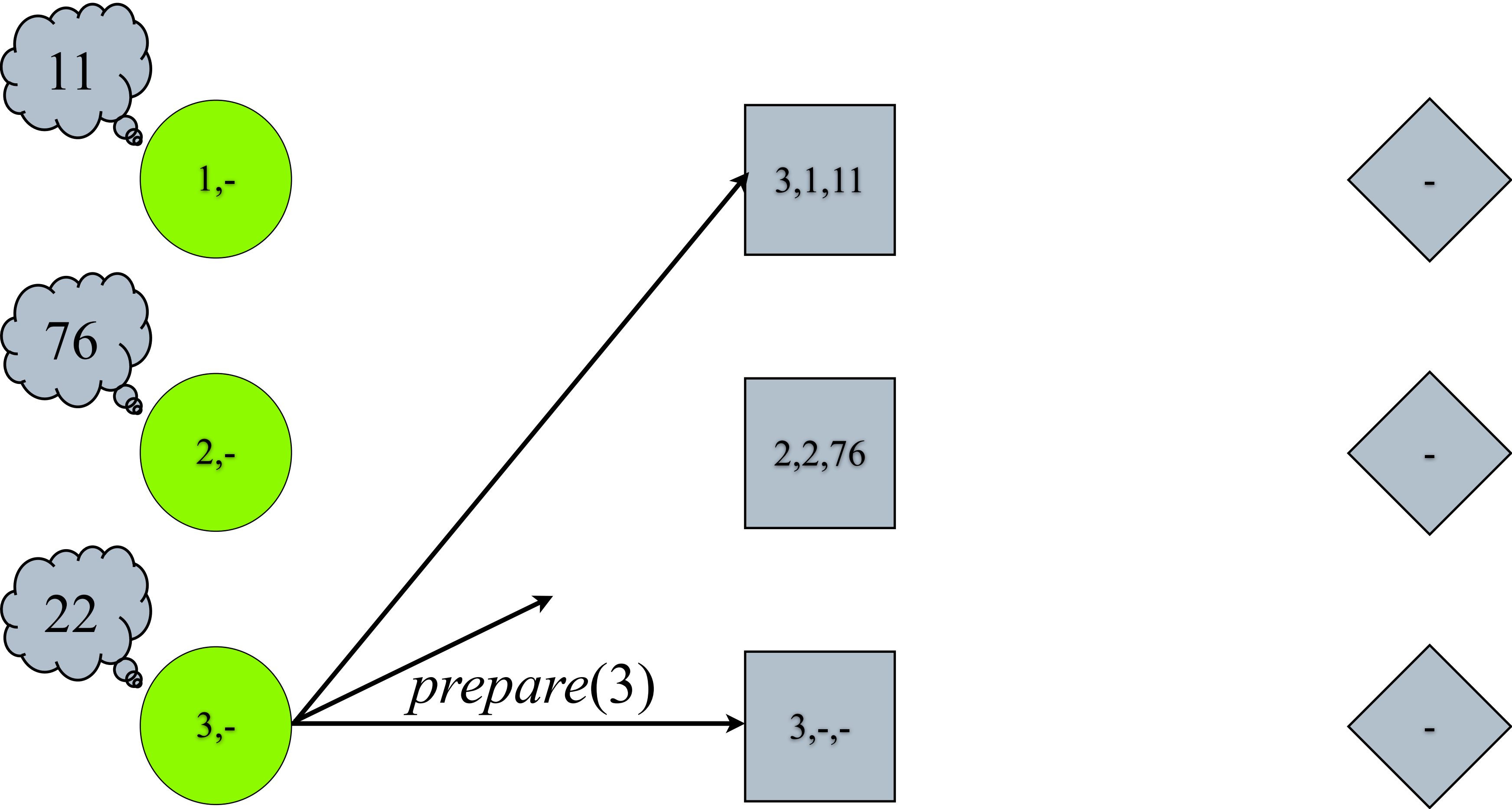
Concurrent Leaders



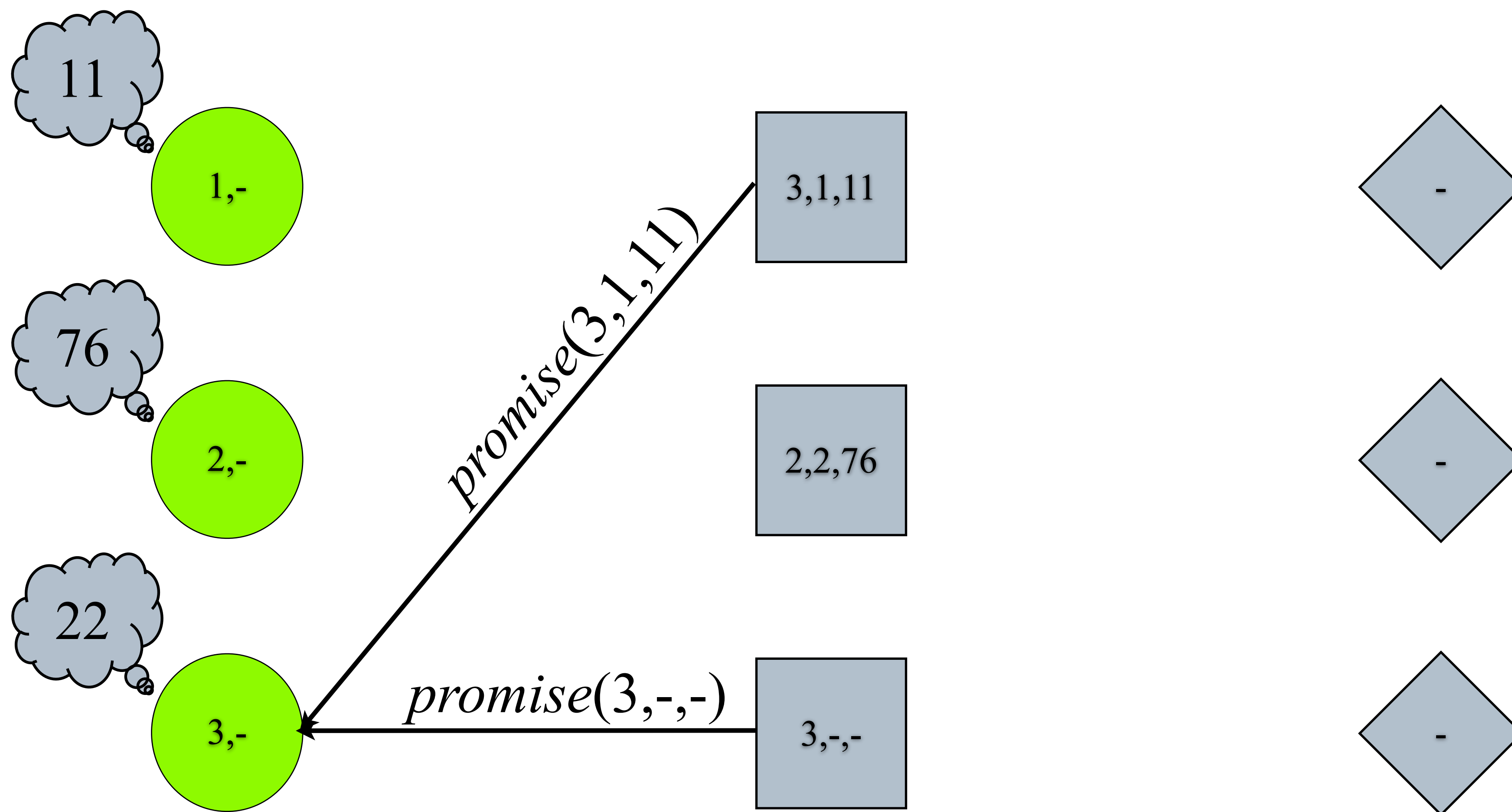
Concurrent Leaders



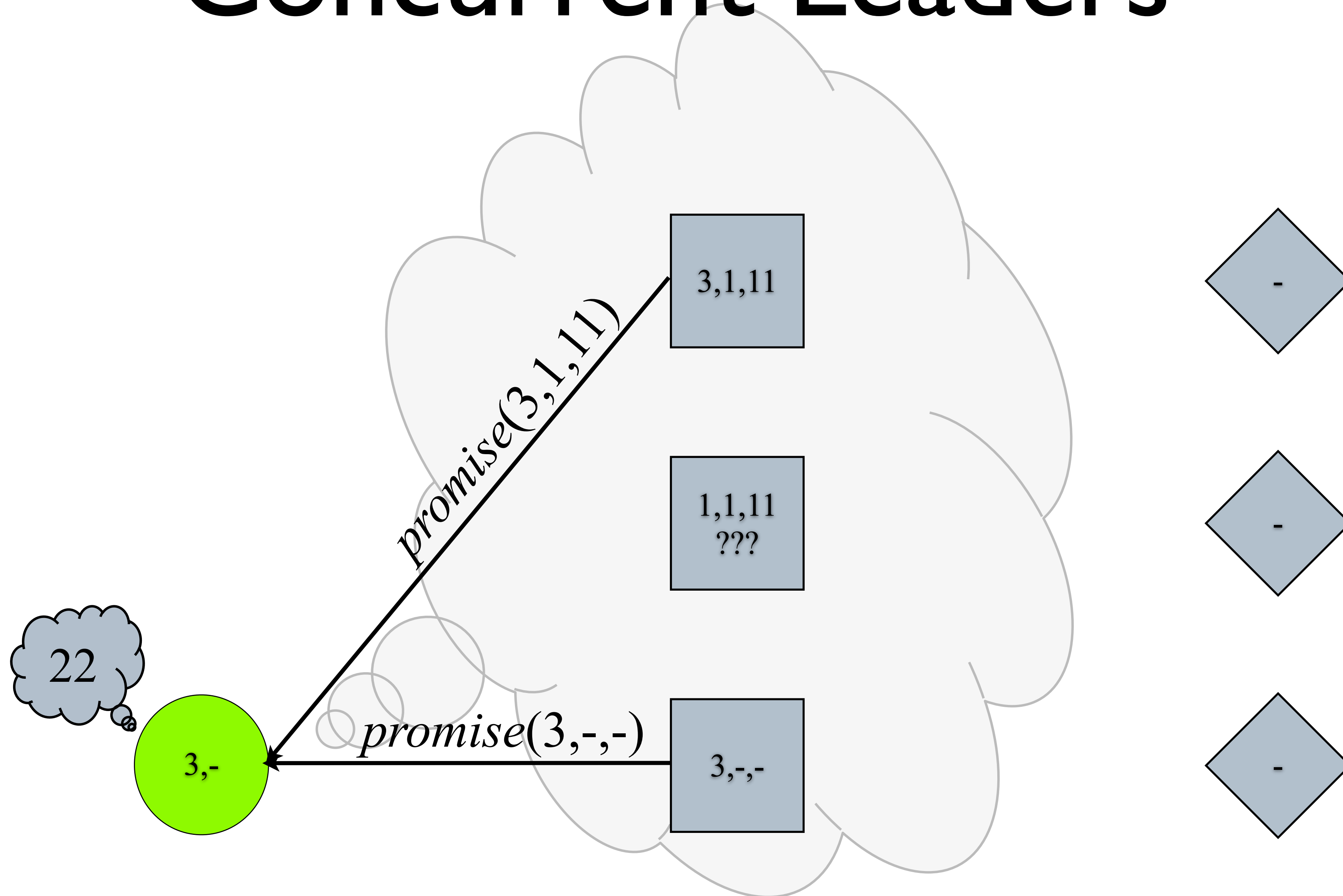
Concurrent Leaders



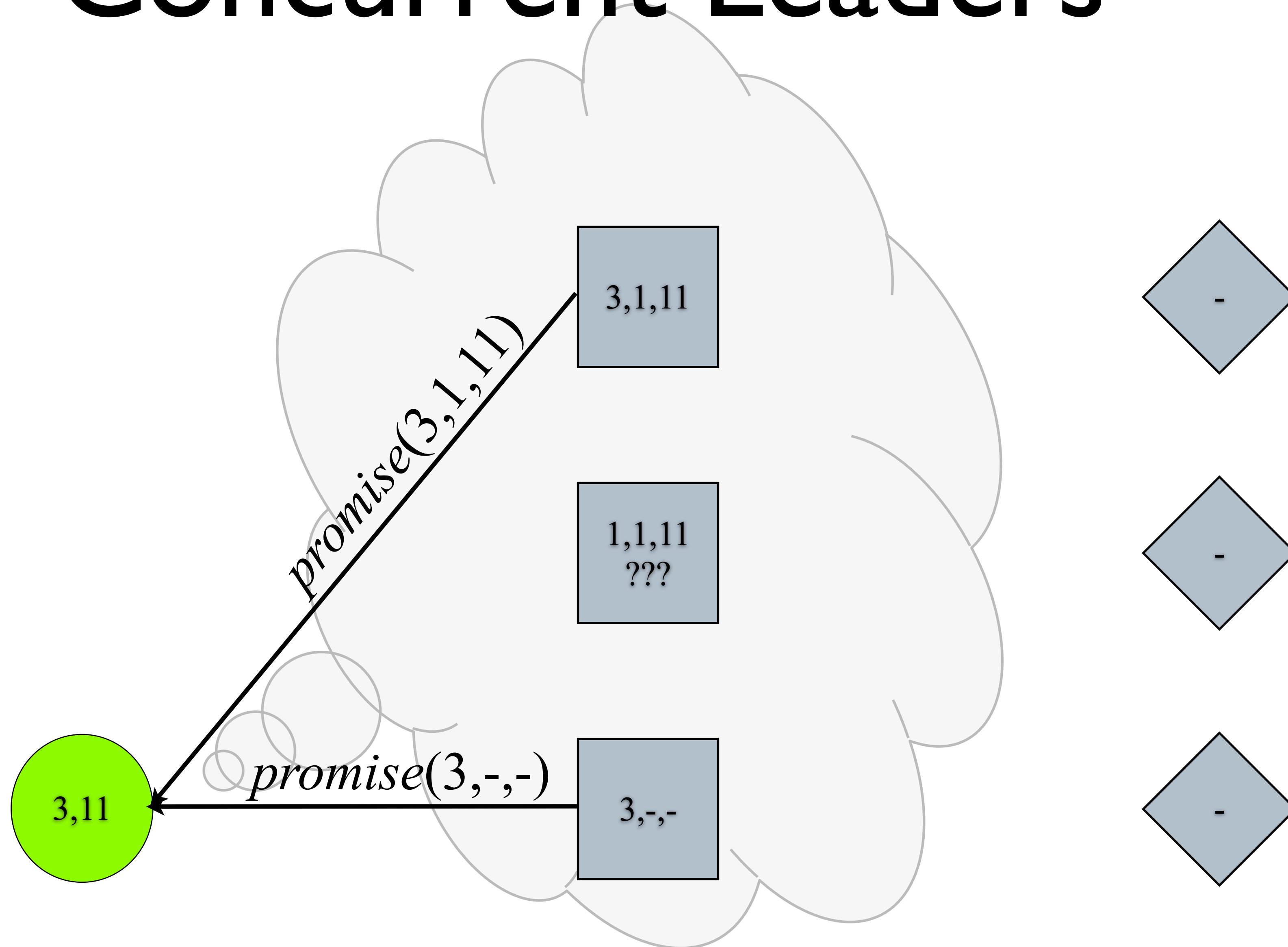
Concurrent Leaders



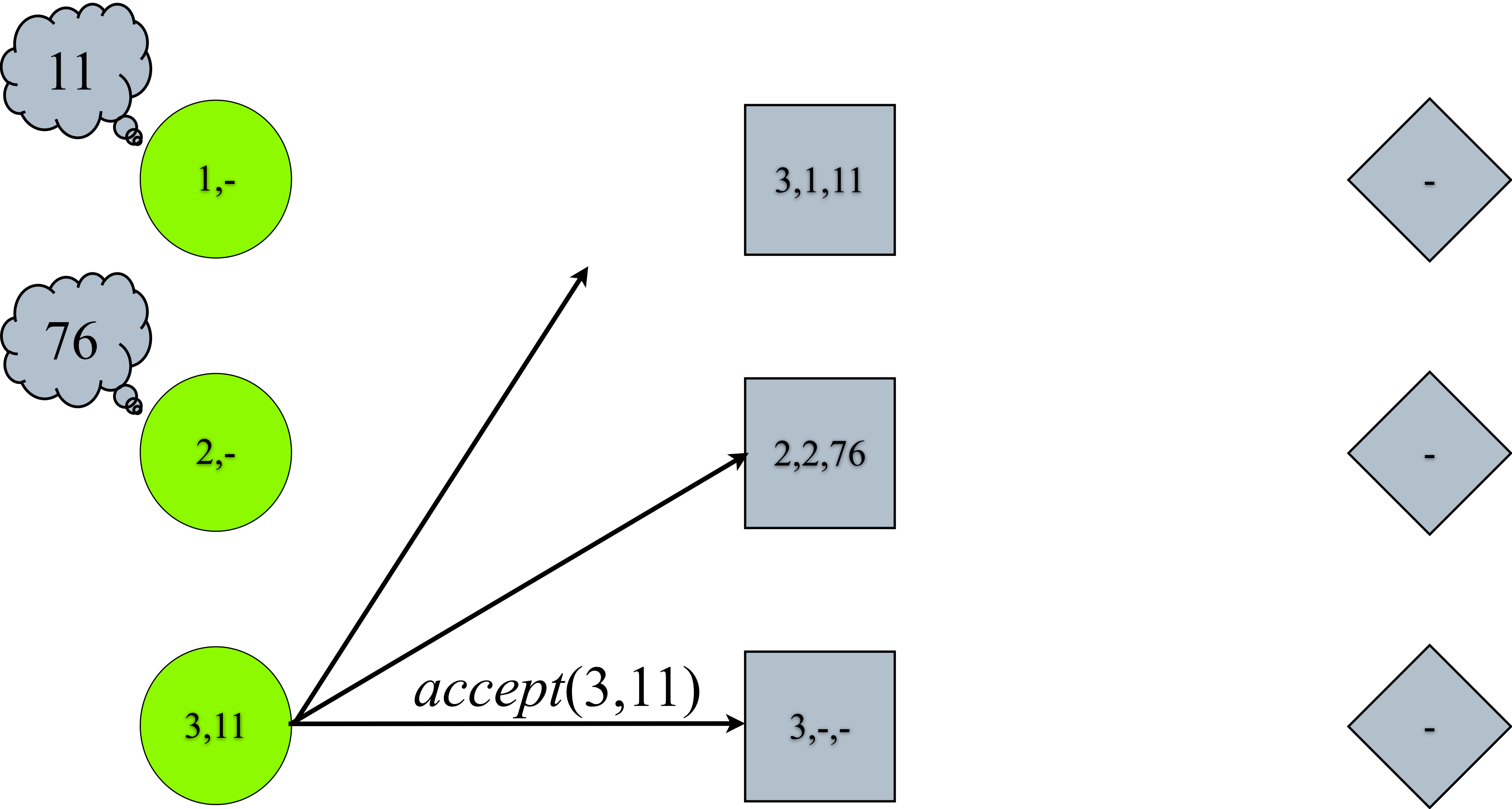
Concurrent Leaders



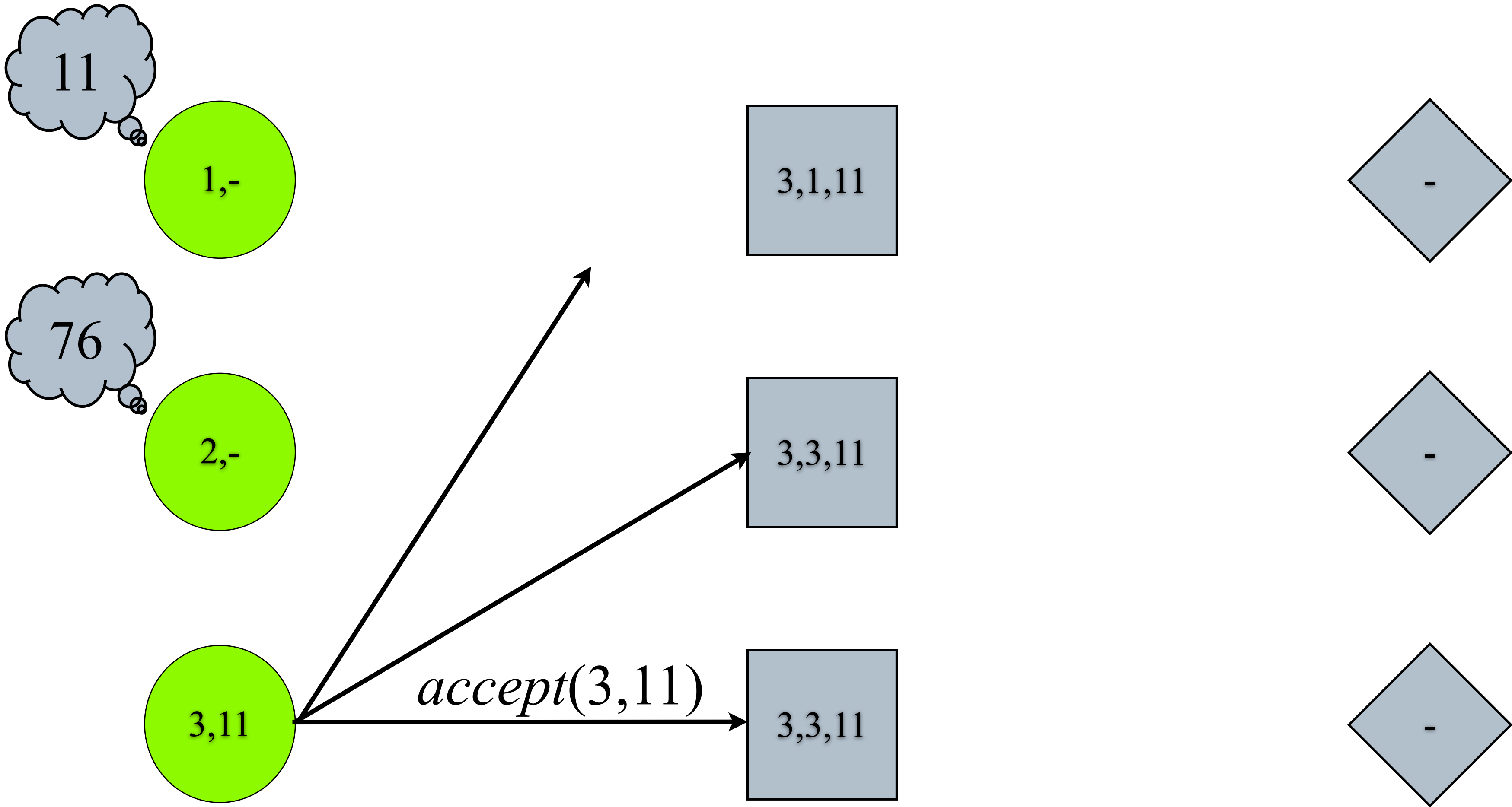
Concurrent Leaders



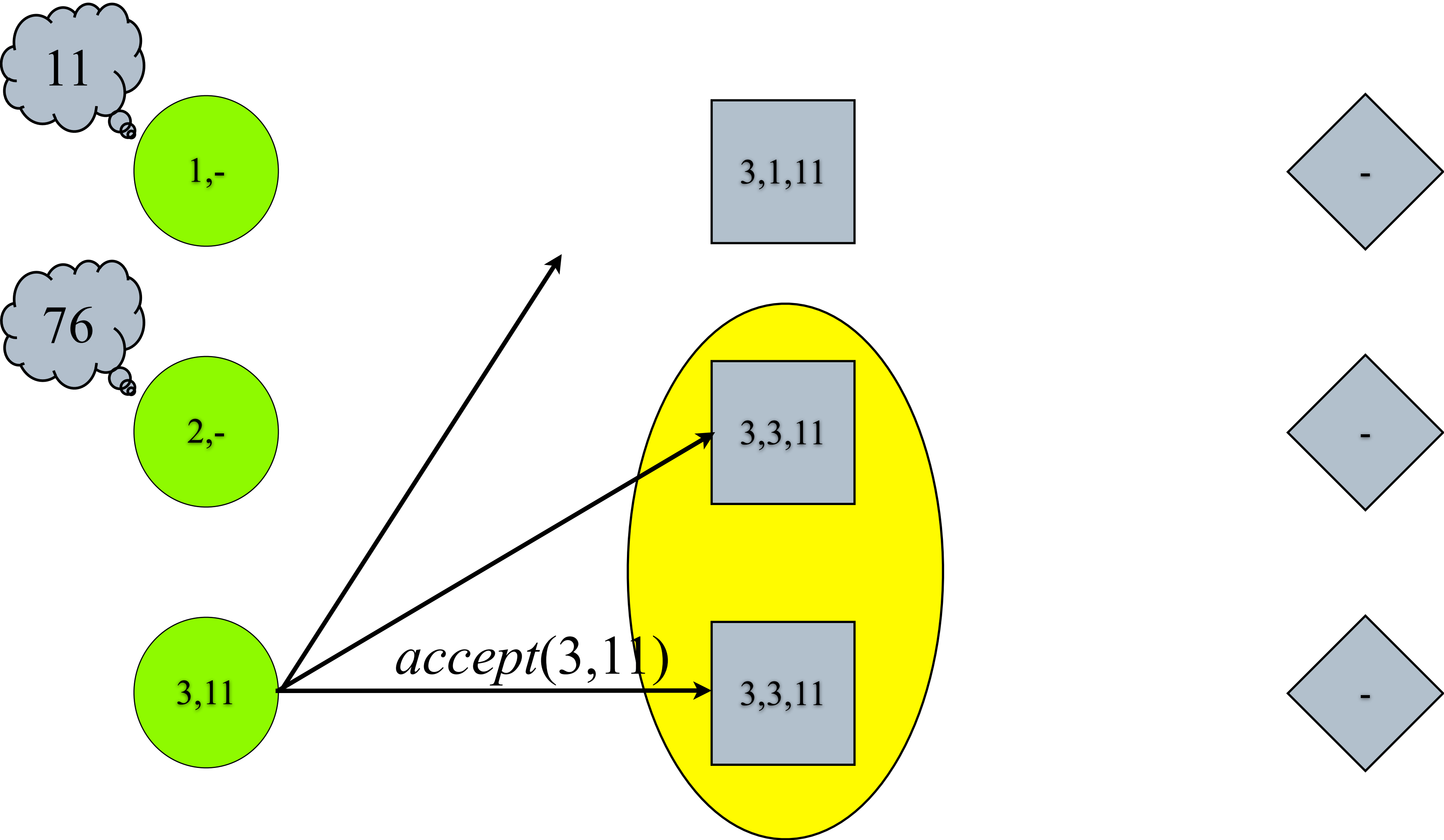
Concurrent Leaders



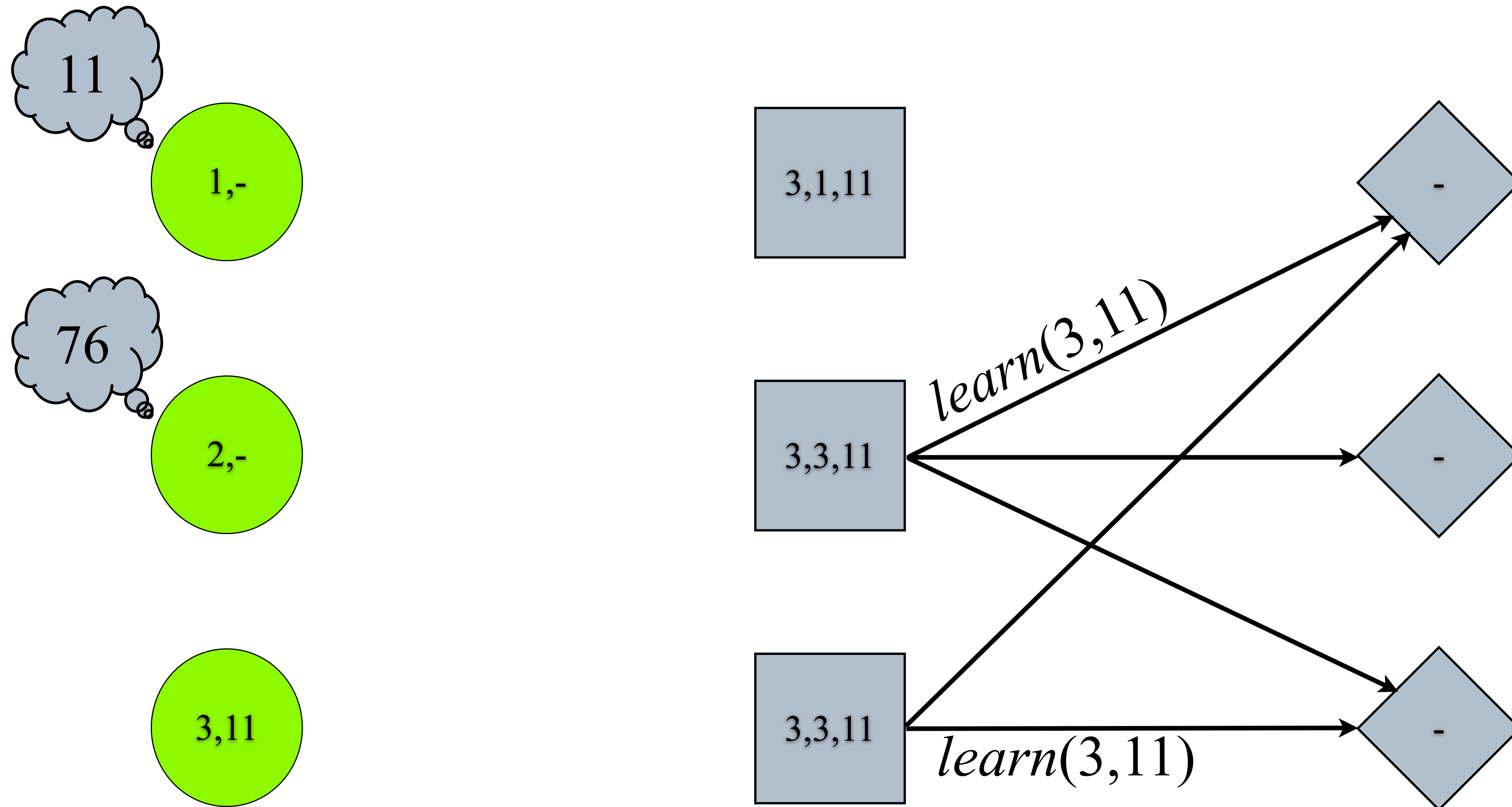
Concurrent Leaders



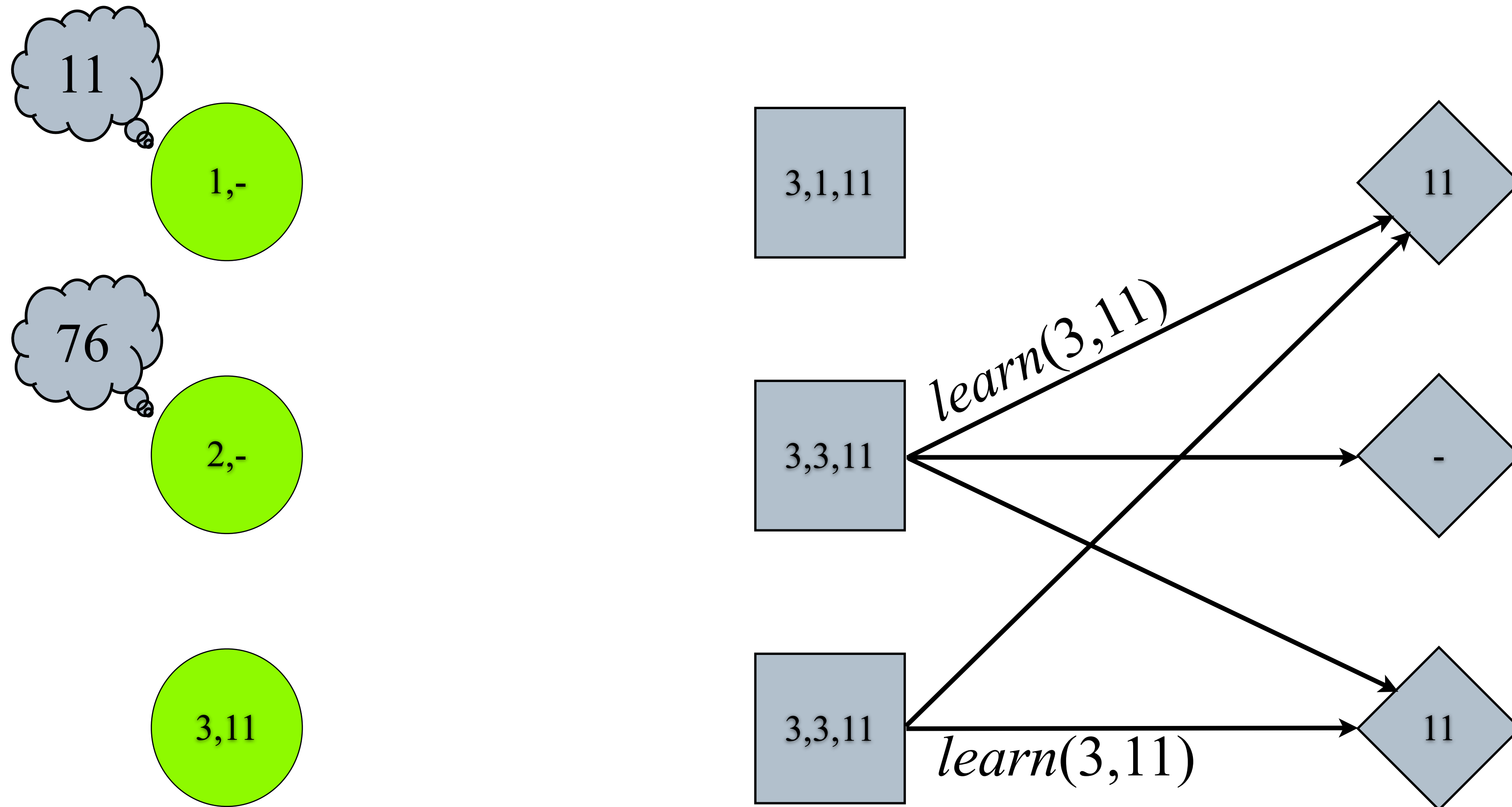
Concurrent Leaders



Concurrent Leaders

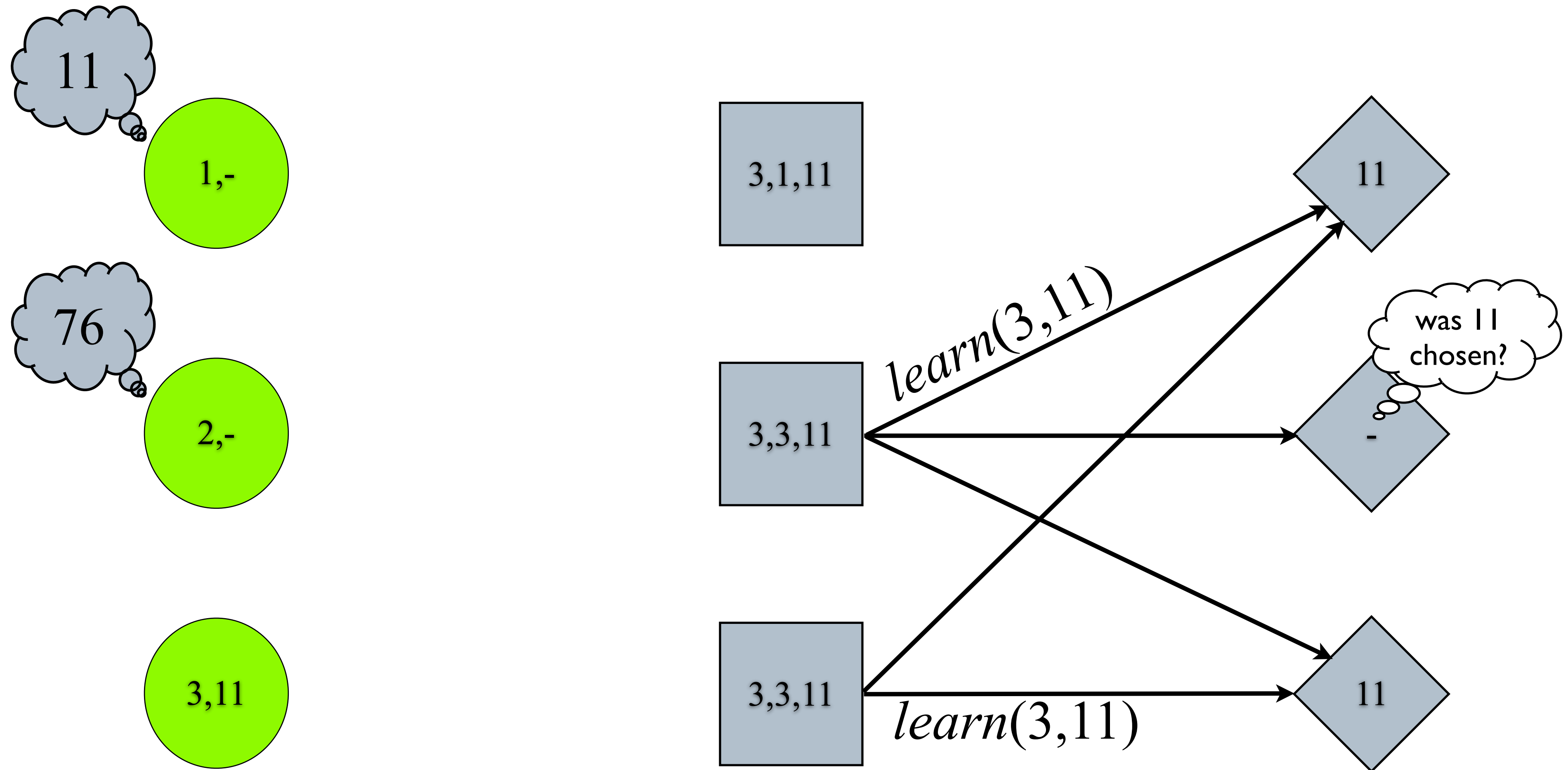


Concurrent Leaders

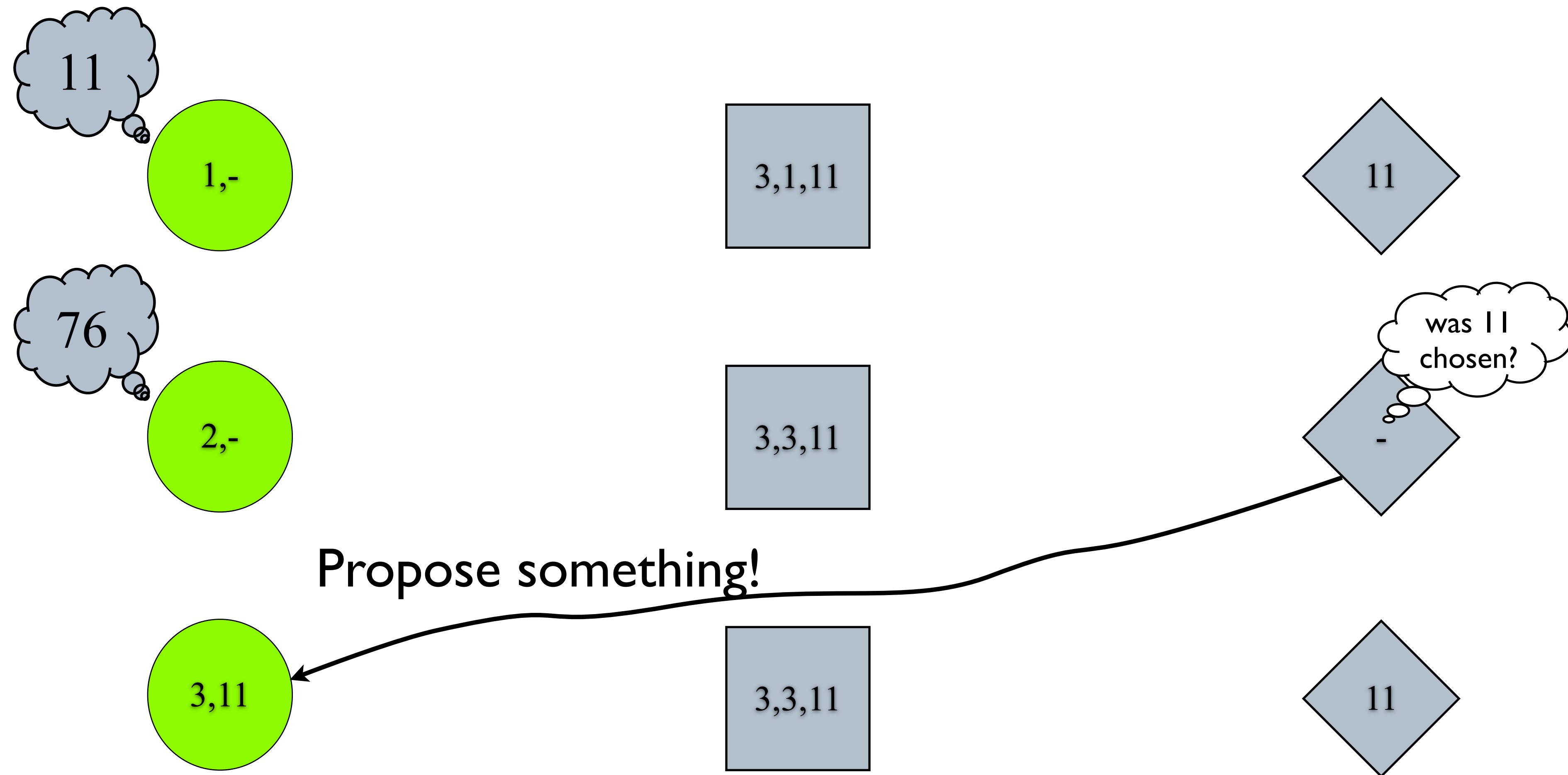


Missing Learns

Learner may not know that a value has been chosen

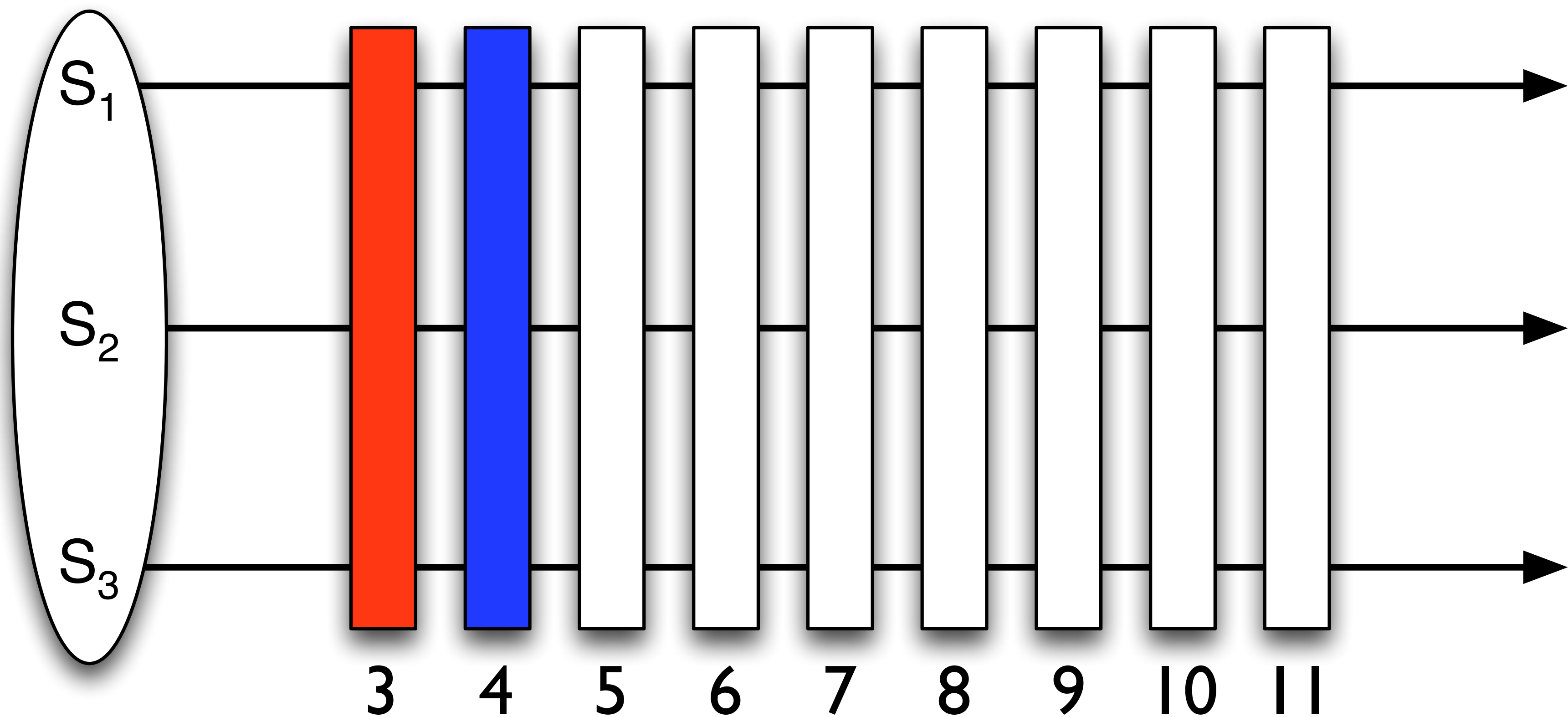


Learner may not know that a value has been chosen



Slots and Concurrency (Pipelining)

Sequence of Slots

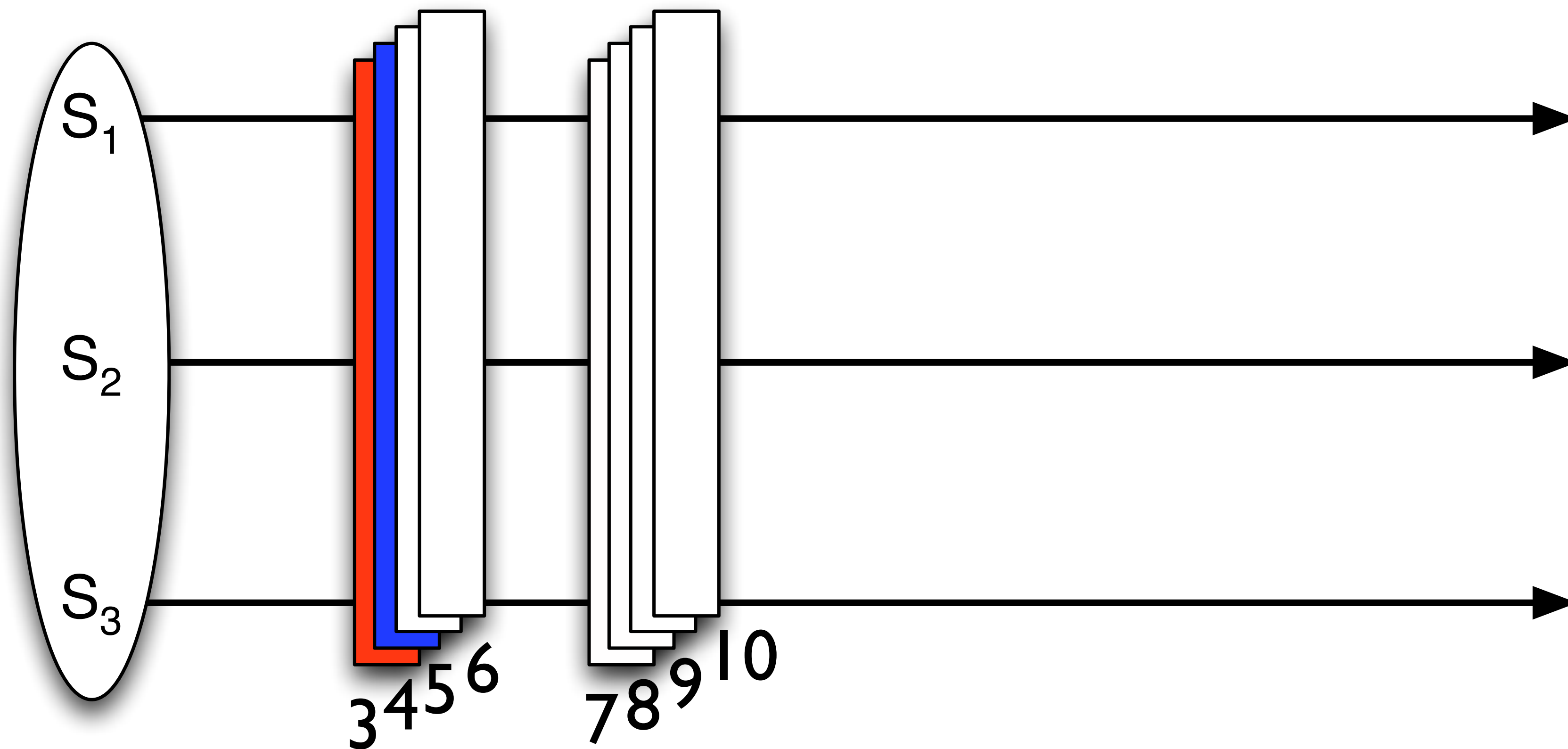


Concurrent Paxos

Executions: Pipelining

- A *Proposer* can start multiple consensus executions without waiting for the first to complete
- This optimization is called pipelining
- To keep track of the different consensus executions, we can use the slot number

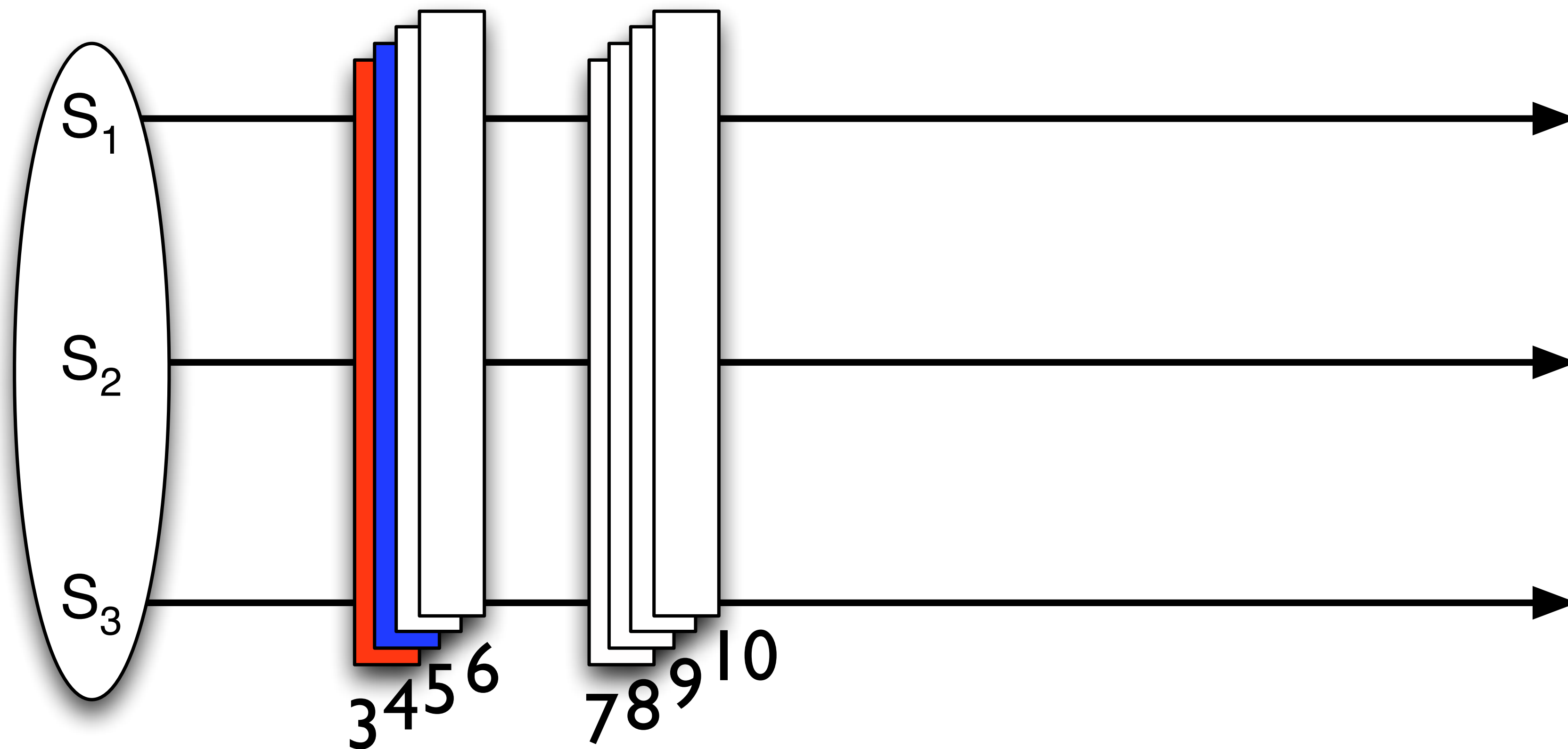
Multiple Slots Concurrently



Concurrent Paxos Executions:

- We want to limit the number of concurrently executing consensus instances (also referred to as slots)
- This limit is denoted with α
- It is called the *pipelining parameter*

What is α here?



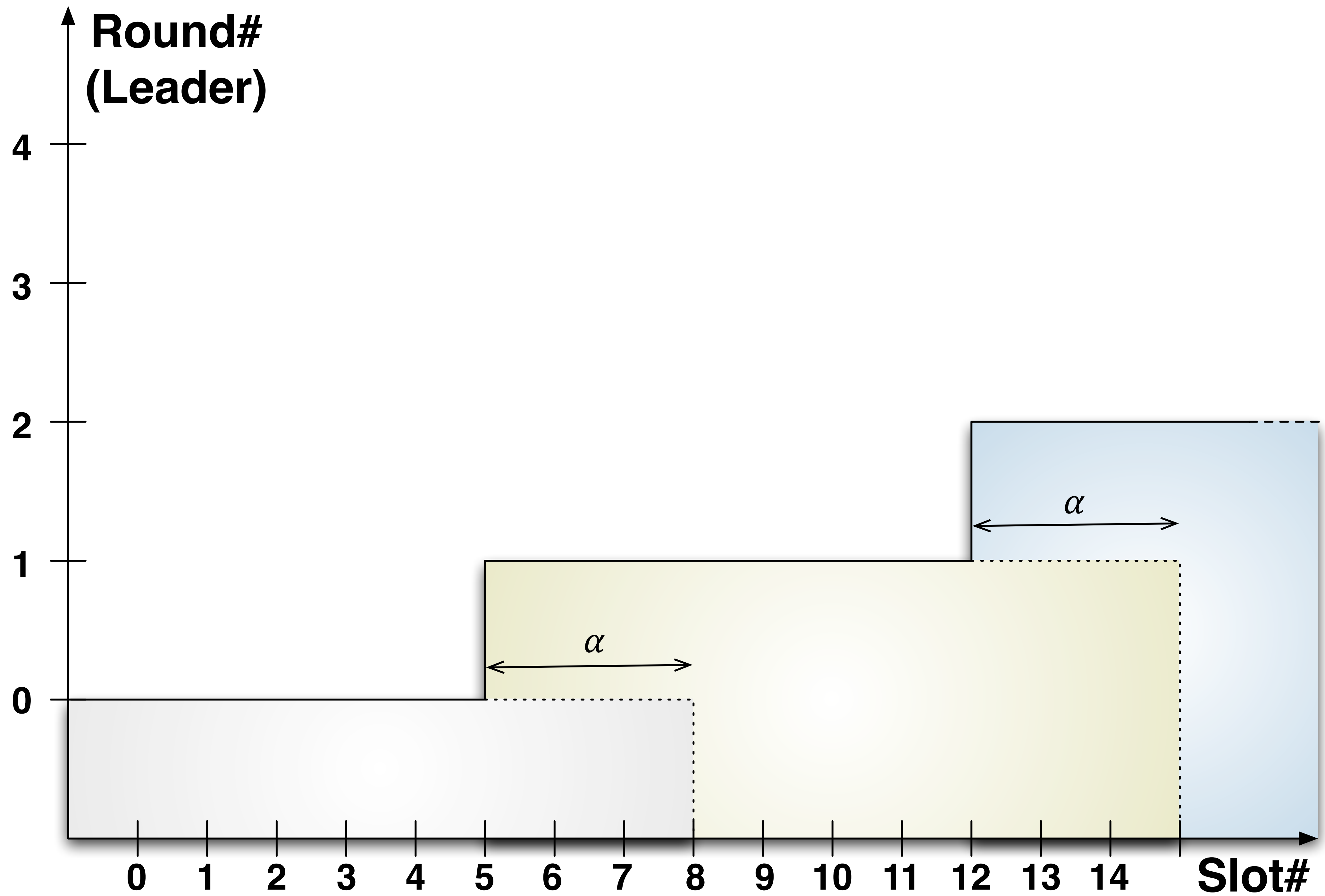
Concurrent Paxos

Executions: A Caveat

- While the proposer can start many pipelined consensus instances
- Commands must be executed in order
- That is, we must wait for the next command in the slot number space

Understanding Paxos

Rounds and Slots



Batching

Batching:

Another Optimization

- A *Proposer* can bundle together many state machine commands in a single consensus instance
- The replicas can execute many commands in the order defined by the *Proposer* without running a separate consensus instance for each of them

Batching: Caveat

- Proposer must wait for
 - K client commands, or
 - some predefined batching timeout, T_B .

Paxos Algorithm

Algorithm 3 Classic Crash Paxos — Proposer c

```
1: Initialization:
2:  $A$  {Set of acceptors}
3:  $crnd \leftarrow 0$  {Current round number}

4: PHASE 1a: Proposer  $c$  (Leader):
5: on  $\langle \text{TRUST}, c \rangle$  from  $\Omega_c$  { $\Omega_c$  indicates proposer  $c$  as the leader}
6:    $crnd \leftarrow \text{pickNext}(crnd)$  {Select proposal number larger than  $crnd$ }
7:    $MV \leftarrow \emptyset$  {Initialize set of (round, vote value) pairs}
8:   send  $\langle \text{PREPARE}, crnd \rangle$  to  $A$ 

9: PHASE 2a: Proposer  $c$  (Leader):
10: on  $\langle \text{PROMISE}, rnd, vrnd, vval \rangle$  with  $rnd = crnd$  from acceptor  $a$ 
11:    $MV \leftarrow MV \cup (vrnd, vval)$  {Add value of acceptor  $a$ }
12:   if  $|MV| \geq n_a - t_a$  then {Got promises from all correct acceptors?}
13:     if  $(vrnd = \perp) \ \forall (vrnd, vval) \in MV$  then {No promises with a value?}
14:        $cval \leftarrow \text{pickAny}()$  {Propose any value}
15:     else
16:        $cval \leftarrow \text{pickLargest}(MV)$  {Pick proposed value  $vval$  with largest  $vrnd$ }
17:     send  $\langle \text{ACCEPT}, crnd, cval \rangle$  to  $A$ 
```

Algorithm 4 Classic Crash Paxos — Acceptor

- 1: **Initialization:**
 - 2: P {Set of proposers}
 - 3: L {Set of learners}
 - 4: $rnd \leftarrow 0$ {Current round number}
 - 5: $vrnd \leftarrow \perp$ {Last voted round number}
 - 6: $vval \leftarrow \perp$ {Value of last voted round}

 - 7: **PHASE 1b: Acceptor a :**
 - 8: **on** $\langle \text{PREPARE}, n \rangle$ with $n > rnd$ from proposer c
 - 9: $rnd \leftarrow n$ {The next round number}
 - 10: **send** $\langle \text{PROMISE}, rnd, vrnd, vval \rangle$ to c

 - 11: **PHASE 2b: Acceptor a :**
 - 12: **on** $\langle \text{ACCEPT}, n, v \rangle$ with $n \geq rnd \wedge n \neq vrnd$ from proposer c
 - 13: $rnd \leftarrow n, \quad vrnd \leftarrow n, \quad vval \leftarrow v$
 - 14: **send** $\langle \text{LEARN}, n, v \rangle$ to L
-

Glossary

- Proposers = Leaders
- Learners = Commanders
- Round = Ballot
- Slot = Consensus Instance
- Prepare = p1a msg
- Promise = p1b msg
- Accept = p2a msg
- Learn = p2b msg

Paxos Properties

Safety

Replicas always remain consistent with each other,
no matter how many crashes occur.

Liveness

If a majority of replicas can communicate with each other,
Paxos can make progress.

Summary

- Paxos needs
 - $2f+1$ replicas to tolerate f failures
 - Two communication steps
- It may not terminate, but it is always safe

